具身智能实验结果与改进方向报告

模型能力 Harness 与推理性能

v4.0 2026 年 10 月 16 日

这份报告把已有机器人操作实验放在一起，解释模型判断、Harness 组织和运行耗时怎样共同影响任务结果。实验从 Astra 的直接抓放开始，经过 GPT-as-Policy 的逐块审核，再进入 RPent 的技能编排；RPent 和 DeepSeek 的计时记录用于进一步解释同样完成任务为什么会花不同的时间。

已有数据指向几项具体工作：抓放需要更可靠的状态判断与恢复；逐块审核和重复技能调用需要按实际贡献调整；仿真吞吐优先处理渲染，而连续控制需要缩短动作块交付时间。云端请求则按工作负载区分：一组主要等首输出，另一组主要等长思考和工具输出。下文先展示每项实验的条件和结果，再分析结论，最后安排相应的改进实验。〔L02–L10，E3 综合分析〕

正文按问题连续展开。附录另列 PPT 内容拆分：同一实验可以用结果、过程和改进方案分别讲解，图表以编号引用。

证据标记：E1 为源码机制；E2 为配置与协议；E3 为报告转引、统计复算、论文结果或明确标注的实验设计；E4 为外部转述。本报告采用公开报告与脱敏数据，累计时间按各实验定义统计。导师提供的 low think 结果标为 E4。

实验	已有数据	本报告关注的问题
bowl 抓放	首批多模型及第二批四组同 seed 对照	定位 抓持判断 恢复选择
GPT-as-Policy	18 格结果与 9 回合 610 轮记录	审核是否纠正动作 接管是否有效
RPent 普通与 Flash	Object Swap 和 Goal 首跑 重试	固定流程 在线规划 失败恢复的取舍
RPent Astra 与 Sol	正式 120 集及独立 8 集深测	同任务效率 渲染 请求与重复动作
DeepSeek	16 集两批对照 单局阶段表 9 体 42 次重放	思考档位 TTFT TPOT 上下文和输出长尾
JEV 与融合训练	本地规则基线 源码审查 Motus 工程记录	选择器分工与后训练实验起点

各批次分别保留任务、初态、成功判据和分母。来源 L01–L16、S1–S7〔E3〕。

阅读顺序：测评任务 → 模型能力 → Harness 对照 → RPent 与 DeepSeek 计时 → 后续改进。附录 A 为 PPT 内容拆分，附录 B 为数据索引与来源。

1 测评任务与实验范围

1.1 RoboDojo 和 LIBERO 分别测什么

我们从 Astra 的机器人操作表现进入这一方向，随后开展本地抓放、GPT-as-Policy 和 RPent 实验。三组实验分别把模型放在直接输出末端目标、审核动作块和选择技能的位置上。因此，报告要同时回答三件事：模型在哪里做对了判断，Harness 怎样把判断变成动作，整条执行链又花了多少时间。两套测评基准分别帮助我们了解能力覆盖，以及在固定任务中比较方法。〔L01，架构依据 E1；研究问题归纳 E3〕

表 01

基准	任务与代表问题	怎样用于本研究
RoboDojo	42 个仿真任务：泛化 12、记忆 6、精细操作 8、长程 8、开放指令 8；另有 18 个实机任务。例：记住输送带上消失的物体、窄孔插管、多物体分类与交接。	观察能力覆盖与外部进展；后续检验本地改进能否迁移。
LIBERO	Spatial、Object、Goal 各 10 个任务，LIBERO-100 再分 90 个预训练任务和 10 个下游长任务，共 130 个。分别改变空间关系、对象、目标与任务长度。	为 GPT-as-Policy 和 RPent 提供环境基础，适合固定任务与初态比较。

来源 W01、W03、W04（E3）。常见的 Spatial/Object/Goal/Long 共 40 任务评价是原有任务的选集；本地 bowl 是自建 robosuite 抓放协议。RoboDojo 与 RoboTwin 是不同基准。

RoboDojo 同时报告终局成功率 SR 和包含中间进度的 Score，仿真总分按五个能力维度平均。这个设计提醒我们：‘没有完成’仍可能来自完全不同的环节。后文会保留抓取、搬运、释放和恢复过程，而不只列一个成功率。〔W01，E3〕

表 02

官方加入日期	模型或系统	Score	SR
2026-09-16	GPT-6-Astra	28.97	22.48%
2026-09-28	PhysicalRSI（Agent 基线）	36.27	31.38%
2026-10-08	VPP2	39.26	32.26%
2026-10-14	EmbodiedEvo（Agent track）	45.08	38.95%

W02 官网首页及 News，2026-10-16 核验〔E3〕；这是不同提交方案的快照，各自的训练、接口与预算条件保留。

近期榜单既出现新的策略模型，也出现组织记忆、代码和反馈的 Agent 系统。对我们最有用的启发，是把基座能力与系统组织一起研究。外部成绩给出方向，解释本地差异仍要回到相同任务、初态和接口下的实验。〔W02、L01，E3〕

2 模型能力 从抓放结果到逐轮判断

2.1 Astra 完成更多抓放任务 平均决策数也更少

bowl 实验要求模型根据三路 RGB 图像和机器人实测状态，把红块放入蓝碗；模型自行决定位置、夹爪开合与结束时机，控制器执行 move_to。观测不提供物体真值坐标，统一上限为 20 次决策、900 个控制步。这个接口把视觉定位、执行进度判断和失败恢复同时交给了模型。〔L02，协议 E2；职责解释 E3〕

表 03

9 月 16 日批次	自动入碗	平均决策数	平均控制步	停在阶段 0–1
Astra 首批	5/5	12.2	432.2	0/5
GLM-5.3-Flash	0/20	18.5	623.1	17/20
DeepSeek-v4.1-flash	1/20	18.1	715.4	17/20
Qwen3.8-flash	4/20	18.9	675.3	14/20

L02 §3.1–3.3〔报告转引 E3〕。Astra 首批为 seed 2000–2004，其余为 2000–2019；阶段 0=未接触，1=接触但未抓起。原始请求档位 medium；Qwen 实际 xhigh，GLM/DeepSeek 后端档位未暴露。Astra 首批独立物理复核为 5/5 达 stage 4；正式盲评只完成两轮，均处于 3–4 边界。

收窄到四家共同的 seed 2000–2004，自动入碗依次为 Astra 5/5、Qwen 1/5、GLM 0/5、DeepSeek 0/5。Astra 用 10–14 次决策完成这些任务；三家各 20 次任务的基线合计有 48/60 停在阶段 0–1，占 80%。差距主要在建立抓取之前就出现了，后续应先查定位与抓持判断，再讨论更复杂的任务规划。〔L02 §3.1–3.3，E3〕

表 04

9 月 23 日配对批次	自动入碗	平均决策数	平均控制步	平均回合墙钟/s
Astra/medium	5/5	16.0	505.8	336.11
Sol/medium	2/5	18.2	622.0	547.09
Sol/ultra	3/5	17.6	732.6	600.63
Luna/max	0/5	19.6	562.0	1969.76

L02 §3.6（报告转引 E3；冻结初态与预算 E2）。均为 seed 2005–2009、串行；Luna 有效末轮为 trial-10-clean。自动入碗样本的录像审核均停在 3–4 边界，Luna 未盲评。墙钟含模型等待、工具往返与环境执行。

第二组同初态结果延续了这一观察：Astra 完成五轮，且平均决策数最少。Sol 增加推理强度后由 2/5 变为 3/5，但逐场景发生了交换：ultra 新完成 2005、2009，却在 medium 已完成的 2007 失败；2008 两档都成功。同时平均控制步由 622.0 增到 732.6，墙钟由 547.09 秒增到 600.63 秒。两档运行产生了不同的成功场景和动作路径。下一轮应在这些初态上重复对照，逐步检查定位、抓持确认和恢复选择，找出增加的思考改善了哪些判断。〔L02 §3.6，E3〕

2.2 Astra 怎样确认抓持并修正错误

Astra 首批 trial-03 提供了一条完整的解释链。第 3 次决策时，指令目标高度为 0.845 m，实测仍是 0.932 m；它识别出前一动作只执行了三个控制步，继续下降。这一步处理的是‘机器人实际到了哪里’，避免把已发出的指令当成已经发生的状态。〔L02 §4.5，轨迹转引与解释 E3〕

随后它也抓空了。第 6 次闭合的目标高度是 0.825 m，第 7 次先试举，第 8 次看到方块留在桌上，于是张爪，把下一次抓取目标降到 0.805 m，并小幅修正横向位置。第 9 次原地闭合、第 10 次再次举升，第 11 次确认方块随手离桌后才搬运。它把试举变成一次状态检验，再用检验结果决定是否恢复；最终用 13 次决策、447 个控制步完成入碗。〔L02 §4.1、§4.5，E3〕

表 05

决策位置	观测与动作	用于判断什么
第 3 次	目标 z=0.845m，实测 z=0.932m；继续下降	指令发出后是否真正到位
第 6–8 次	在 z=0.825m 闭合并试举，物体仍留桌面	抓持是否建立
第 8–10 次	张爪，目标降至 z=0.805m，横向微调后重抓	根据失败结果改变下一动作
第 11 次	确认物体随手离桌后搬运	恢复是否有效

Astra trial-03 逐轮摘要。L02 §4.1、§4.5（E3）。原报告附完整图像与轨迹。

trial-04 则显示了方向纠错：第 4 次决策发现上一轮横向移动使腕视偏差扩大，便反向修正 y 到 0.045 m。该轮首次抓空后也经过试举、下降重抓和随动确认，最终在 14 次决策内完成。两条轨迹都出现过初始误差，但模型随后利用新观测修正方向或抓取高度，把任务继续推进到搬运和释放。〔L02 §4.5, E3〕

对照案例把问题说得更具体。DeepSeek trial-01 依据夹爪中间开度判断已经持物，物理复核却显示全程未抓住；GLM trial-14 在未接触方块时报告完成。Qwen trial-13 则成功用手与物体的同步像素位移验证抓持，并完成入碗。由此，Astra 的本地优势更适合概括为较稳定地串起‘定位—检查—纠正’，这些组成能力也能在其他模型的成功轨迹中观察到。〔L02 §4.4–4.5，E3〕

2.3 把空间能力拆成三个可测环节

这些轨迹需要分别检查定位、状态判断和恢复选择。定位看对象、方向与落点是否正确；状态判断看模型能否分清未到位、已接触、已抓住和已完成；恢复选择看它发现偏差后是否采取了有效动作。这样才能解释一次失败究竟始于看错目标、误判状态，还是选错了下一步。〔L02，能力归纳 E3〕

表 06

能力问题	下一组配对实验	关键读数
定位或参考系理解	原输入与增加物体真值位置的诊断输入比较；其余相机、接口、预算固定	目标误差、错误修正方向率、首次举升轮次
动作后状态判断	相同帧数与视角下，比较重复当前帧和动作前后关键帧	假抓持率、假完成率、状态识别准确率
执行时序适配	原接口与统一到位等待、夹爪时序的接口比较	到位前重复调用、抓取控制步、成功数
恢复选择	在同一抓空或滑落状态重置，比较继续执行与验证后恢复	恢复成功率、额外调用数、最终成功数

依据 L02 的失败复核与工具时序提出，均为待实施设计〔E3〕。诊断真值仅用于定位问题；部署输入保持实际可取得的观测。

这些实验也能分开模型能力和接口负担：原报告中，同样约 10 cm 的移动，是否携带夹爪目标可使执行窗口从 3 步变成 40 或 80 步。若统一时序后重复调用减少，优化应落在接口；若补入位置或前后帧后抓持错误减少，则优先补相应训练数据。先做这一轮诊断，再决定把训练预算花在哪里。〔L02 §1、§5, 机制转引及实验推导 E3〕

2.4 进入 RPent 后 模型面对的问题变了

在 RPent 中，planner 可以利用任务记忆调用技能，目标定位由 SAM3 支持，动作由 π0.5 技能和运动原语执行。它的主要工作因此转为选对象、选技能、检查阶段和处理异常。抓放实验要求模型自己连续决定末端落点，RPent 则把其中一部分工作交给专门模块。〔L01，架构 E1；职责归纳 E3〕

表 07

RPent 套件	Astra/medium	Sol/medium	值得继续解释的差异
Spatial Task	19/20	17/20	目标选择与具体执行路径
Spatial Swap	20/20	20/20	同样完成时，各自用了多少调用和动作
Long Swap	11/20	12/20	阶段衔接、恢复与长程错误累积
总计	50/60	49/60	成功数接近，进一步比较过程成本

L04（报告转引 E3；配置与验收 E2）。每模型三套件，各 10 任务×2 初态；使用冻结环境终态判据。bowl 与 RPent 的任务、接口、预算和成功判据不同，两组之间不是 Harness 的因果消融。

Spatial Swap 两模型都完成 20/20，平均回合耗时仍分别为 586.9 和 943.7 秒，说明相同成功数可以对应不同的运行成本。Long Swap 则需要先查清未完成的阶段。接下来分别检查调用顺序、工具反馈和失败恢复，再把时间拆到请求与执行阶段，判断额外成本从哪里产生。〔L04，E3；墙钟含会话内等待〕

3 Harness 从动作审核到技能调用

3.1 先看决策被放在了哪一层

同一个模型接到不同控制接口，承担的工作会明显变化。GPT-as-Policy 的 Direct 要从图像判断目标、生成末端位姿，再由程序转成短段动作；Student 直接执行 π0.5；Hybrid 则先取得 π 候选，再让 Astra 决定放行、修改或接管。RPent 把一次高层决策扩大到定位、抓取、移动等工具，π0.5 在接触技能内部连续执行，明确的几何运动由解析控制器完成。实验因此同时改变了模型负责什么、多久被调用一次，以及失败后由谁处理。〔L03、L06、L09；E1 接口机制，E3 比较〕

表 08

路线	模型交付什么	动作执行与反馈
Student	π0.5 输出连续动作块	执行前缀后重新观测、采样
Direct	Astra 输出末端目标、姿态和夹爪命令	边界检查后执行 1–5 步
Hybrid	Astra 审核 π 候选，选择放行、修改或接管	执行后再审核下一段
RPent 普通模式	LLM 选择工具与参数	π 接触技能、解析伺服及工具反馈
RPent Flash	预定流程与锚点重新绑定	视觉定位、π 和解析工具继续运行

L03、L06、L09、L10；接口机制 E1，Flash 批次配置 E2。下文按各批次任务与协议展示结果。

沿着这条分工变化看数据，可以回答三个具体问题：每次动作都审核，究竟纠正了什么；把动作交给技能后，剩余失败发生在哪里；进一步减少高层往返时，哪些检查必须留在执行端。下面先从观察最完整的 GPT-as-Policy 两批实验开始。〔E3〕

3.2 18 格实验暴露了接口误用与目标识别问题

9 月 21 日的 18 格由两个任务、Ori／Task／Pos 三种条件和三条路线组成。Task 改动后同步更新了输入指令：碗换成葡萄酒瓶，字母汤罐换成奶油奶酪；Pos 保留指令、改变位置。这使我们能够区分指令改动与空间布局改动，而不是把所有失败统称为泛化不足。〔L17，协议 E2〕

表 09

条件与目标	Student	Direct	Hybrid
Goal Ori：碗放盘上	成功／75 步	失败／300 步	失败／0 步
Goal Task：酒瓶放盘上	成功／104 步	成功／236 步	成功／129 步
Goal Pos：碗放盘上	失败／300 步	失败／300 步	原生失败／145 步
LIBERO-10 Ori：双罐入篮	成功／272 步	失败／520 步	成功／271 步
LIBERO-10 Task：换目标后入篮	成功／284 步	失败／520 步	成功／416 步

条件与目标	Student	Direct	Hybrid
LIBERO-10 Pos：双罐入篮	失败／520 步	失败／520 步	失败／520 步

L17；结果 E3，配置 E2。每格 init 0、环境 seed 0，Astra medium；同条件初态匹配，π 采样随机状态未配对。主表采用原生判据：Student 4/6、Direct 1/6、Hybrid 3/6。Goal Pos Hybrid 画面经用户接受，原展示列为成功；中心距 36.75mm，原生阈值 30mm。Goal Ori Hybrid 展示为同配置复跑，仍为零步失败。

最先暴露的是接口问题。Goal Ori Hybrid 原跑与复跑都在获取 π 候选后，把 stop 当成查看画面的操作，先提交 steps=0 被拒，再以 steps=1 结束任务，机器人始终没有动作。对此应把只读观察和不可逆终止分成清楚的接口，检查 stop 的描述及返回状态，再单独重跑新协议。增加抓取训练或提高推理速度，都不会直接修复这一类误用。〔L17；E3 轨迹，E3 解释〕

另一类是目标选择。更换奶油奶酪的回合中，Direct 先处理了错误包装，到第 461 步才识别出混淆，520 步预算仅剩 59 步。同条件 Student 在 284 步成功；Hybrid 在 416 步成功，但 84 次决策全部放行学生，没有执行修正。再看位置扰动，六格按原生判据全部失败。数据指向目标绑定、空间定位和动作适配三个可分别检查的环节，也说明“加上审核模型”本身没有保证纠错。〔L17；E3 记录，E3 判断〕

3.3 九回合记录显示持续审核增加了等待

随后采用逐轮 JSON 协议，把每次输入、候选、决策和实际动作对应起来，形成三任务各三路线的九回合记录。它回答的是上一批未能精确拆清的机制问题：审核是在改变动作，还是只给既有动作增加等待。〔L03；E1 协议，E2 批次〕

表 10

任务	Student：结果／步／轮	Direct：结果／步／轮	Hybrid：结果／步／轮
双罐入篮	成功／260／52	成功／476／97	成功／253／51
杯子入微波炉并关门	成功／348／70	失败／520／105	成功／251／51
打开底层抽屉	失败／300／60	失败／300／64	失败／300／60

L03；结果 E3，配置 E2。九回合共 610 轮、3,008 控制步；各格单回合，同任务初态匹配。协议与旧 18 格不同，π 随机采样未配对。

双罐入篮中，Student、Direct、Hybrid 的单轮均值分别为 4.203、14.413、18.818 秒；后两者的 GPT 往返为 10.819、14.374 秒。Hybrid 的 51 轮全部放行 π。微波炉也是 51 轮全部放行，回合墙钟为

1,026.640 秒，纯 π 为 262.337 秒。Hybrid 的控制步较少，但实际没有动作纠正发生，轨迹长度差异还混有 π 采样差异。当前可直接确认的，是每次审核都进入了串行等待。〔L03；E3 数据，E3 分析〕

这种等待还有明确的协议来源。双罐 Hybrid 平均输出 331.8 个可见 Token，首末可见输出间隔约 9.62 秒；即使选择放行，也填写完整 assessment、edit 和 target 结构。因此第一项改动可以只缩短放行响应，保留请求编号、模式和必要执行参数；详细解释按需记录。保持任务、模型、候选和审核频率不变，先测完整决策时延与动作一致性，再改变调用频率。〔L18；E3 数值，E3 设计〕

抽屉回合进一步说明，允许接管与纠错成功是两件事。Hybrid 60 轮中有 13 轮末端接管，但底层抽屉终值仍为 0，中层却被拉到 –0.159787m；纯 π 的中层为 –0.159949m，底层同样为 0。下一项对照应针对“当前操作的是哪一层”加入前后观测核对，并统计接管后的目标变化。只有确认监测能识别偏离，再把固定逐轮审核改成阶段变化、目标丢失或持续无进展时触发，才能知道省掉的审核是否影响成功。〔L03；E3 轨迹，E3 设计〕

3.4 普通模式与 Flash 的完成率和时间取舍

RPent 将抓取交给 π 技能、几何运动交给解析工具，高层依据记忆和反馈选择调用顺序。9 月的 GLM-5.3 普通模式在 Object Swap 首跑完成 18/20，在 libero_goal 完成 16/20；Flash 改为回放任务计划并重新定位锚点，Object Swap 完成 15/20。它们提供了在线编排与固定流程的实际比较。〔L10、L19；报告结果 E3〕

表 11

Object Swap 条件	规划／感知／动作	结果	时间统计
普通 API	GLM-5.3／SAM3／π 与解析工具	首跑 18/20	全 20 集均 890.20s；成功 18 集均 619.22s
Flash	固定计划／Molmo2-8B／π 与解析工具	首跑 15/20	成功 15 集均 228s；规划 LLM Token 为 0

L10、L19〔E3〕。API 逐集表累计 17,804s；全批耗时÷成功数为 989.11s。Flash 的 228s 是 15 个成功回合的均值，未提供全部 20 集时长。两批定位模型与流程不同，Flash 与 goal 曾共享 VLA。

Flash 的五个失败中，两个 t2 初态都因计划写 ranch dressing bottle、任务写 salad dressing 而在约 22 秒中止；另外三个执行完整回放后未达目标，涉及锚点重定位或放置精度。固定流程因此首先需要验证任务与计划是否匹配，其次需要在动作后检查物体是否发生预期变化。可将“对象名无法绑定”直接返回高

层重规划，把定位不确定、抓持丢失和放置失败分开处理。这样才可能保留固定流程的调用优势，同时修补它已出现的失败出口。〔L10；E3 记录，E3 设计〕

普通模式也有另一种代价：Object Swap 两个败局都用满 100 次请求，分别持续 2,818 和 3,840 秒，反复使用定位、反投影和移动原语。libero_goal 的抽屉 t0 两初态首跑也都失败。现场试错使系统有机会适应偏差，但当工具反馈没有明确说明失败阶段时，高层会继续为相似动作支付规划和执行成本。应先检查这些重复中是否改变了目标、接近方向或抓持状态，再决定哪些尝试交给局部恢复。〔L10、L19；E3 记录，E3 解释〕

3.5 重试挽救了失败 也增加了成本

对普通模式六个失败身份各重跑一次，五个转为成功：Object Swap 的覆盖从 18/20 变为 20/20，libero_goal 从 16/20 变为 19/20。两个 Object Swap 败局重跑分别在 601、617 秒完成；goal t5_s1 却从首跑 823 秒失败，变成重跑 4,795 秒成功。特定抽屉布局 goal t0_s0 仍反复失败。这些结果说明，重新开始有时能离开不利动作路径，也说明成功轨迹未必短。〔L10；报告结果 E3〕

表 12

已做的检查	收益	代价或对应现象
六个失败身份重试	新增 5 次成功	新增约 16.9M Token；约 3.4M Token／新增成功
按上下文、decode、请求数阈值回测早停	四个失败集可省 22.2M Token，约全批 21.5%	同时截断 5 个最终成功集；总成功率降低 12.5 个百分点
检查接近请求上限的 10 集	其中 6 集最终成功	请求数高表示任务困难，不能直接判定必败

L10；E3 原批记录，E3 回测结论。重试结果与首跑分列；阈值早停是离线回测，截断后重跑的组合策略尚未实跑。

因此，恢复触发应加入任务证据：相同物体连续抓空、物体没有随夹爪运动，或同一目标下残差长期不降。先在当前场景内重新观察、改变接近方式并限制恢复预算；仍无进展时返回高层。整集重置则作为另一种策略单独比较，把原失败、重试成本和新增成功全部计入。下一轮真正要测的是哪一种恢复改变了失败路径，而不是预设到达某个 Token 数就停止。〔L10、L11；E3 设计〕

3.6 用阶段结果解释长任务失败

后续 120 集将高层换成 Astra 与 Sol，均为 medium，使用冻结任务资产和记忆。Spatial Swap 两模型都完成 20/20；Long Swap 分别只有 11/20、12/20。普通任务中的工具组合已经能完成多数目标，较长的抓取、搬运、插入和关门序列则成为下一步诊断重点。该批普通任务失败不重跑，与前述 retry-once 分开统计。〔L04；报告结果 E3〕

为看清动作过程，另做 t0 双罐与 t9 微波炉的八集深测。t0 中 Astra 两集各使用 15 个 π 动作块并成功；Sol seed0 两次 pick 各用 14 块、均未满足局部验收，但整集成功。Sol seed1 则进行了 8 次 pick、116 块，最终失败，其中后段六次失败 pick 消耗 88 块、268.72 秒。单次 pick 的返回值与整集结果并不等价。下一步需要把 false 拆开：对象未对准、未建立抓持、举升未达阈值，还是验证信息不足，才能选择对应的恢复动作。〔L06；报告结果 E3〕

源码显示 pi0_pick 的验收依据末端下降、随后抬升和夹爪条件。这些量反映动作过程，需要结合物体是否随夹爪移动来判断抓持。Sol seed1 有一次已满足夹爪条件，但抬升只有 0.0222m，低于 0.05m；简单放宽夹爪阈值并不对应这次失败原因。可以先增加 confirmed／lost／unknown 三类抓持状态，保留观测帧、目标关联和判断依据，再测误报、漏报与恢复后结果。〔L06 源码摘录、L11；E1 机制，E3 数值，E3 设计〕

t9 四集都失败。Astra seed0 的 14 次移动耗满预算，共消耗 672.90 秒；Sol seed0 的五次接触工具调用使用 84 个 π 块。这里的工具 success 直接取整任务 terminated，原记录没有分别保存杯子 In 和门 Close，因此能够定位的是“调用未使整任务完成”。下一轮分别记录抓杯、送入、释放撤手和关门的效果，再决定复合技能的边界。〔L06、L11，E1/E3〕

3.7 先改状态检查与恢复 再比较 JEV 选择器

上述结果支持按以下顺序改动 RPent。先让新检查只记录、不控制动作，确认它能区分抓持、空动和未知；随后将同样的状态交给原高层，再增加局部恢复，最后扩大单个技能覆盖的动作范围。这样每次变化都有对应问题：信息是否改善判断、恢复是否挽救失败、技能是否减少往返。〔L11；E3 设计〕

表 13

对照	唯一新增的机制	主要看什么
当前 RPent → 提供阶段状态	抓持与进展证据送给原高层	误判、调用变化、阶段与最终成功
阶段状态 → 局部恢复	固定预算的重观察与重试	挽救数、误触发、重复动作与耗时
局部恢复 → 复合技能	组合现有定位、π、搬运和释放	减少的高层请求与新增局部成本
固定候选下更换选择器	规则／Astra／JEV 分别选同一组候选	选择质量、完整延迟、最终任务表现

L09、L11；E3 方案。先固定 planner、模型档位、π／感知、记忆、初态及预算。诊断先做独立校验；环境子谓词供评分，不加入普通视觉策略输入。各机制冻结后使用留出初态。

JEV 适合放在最后一项选择器对照。embodied-jev 的 LIBERO v2 由 GPT 比较前后画面、提出 2–3 个短动作，Jev 或 GPT 选择候选、速度档或重新观察，实际运动由确定性伺服执行；每个候选最多四段、每段五步，到位或停滞会刷新。它提供的是有限候选决策，不直接生成 π 那样的连续动作块。若接入 RPent，可以让它选择继续、重新定位、换接近方式或返回高层，并保留 π 处理接触动作。〔L09、W18；E1 机制，E3 接入设计〕

本机已经验证的是规则基线：三个内置任务各三个 seed，9/9 成功、模型调用为零，另有 94 项测试通过。Jev 的 HTTP 路径与本地 MiniCPM 候选评分路径完成了源码核查；本机没有据此取得 Jev 模型的任务成功率。首轮可先固定状态与候选做离线规则／Astra／JEV 对照，再把选择结果接回同一执行器。候选生成、额外观察和选择请求的时间都计入，才能判断小选择器是否真的减少了高层等待。〔L09；E3 本地验证记载，E1 路径，E3 设计〕

3.8 把有效经验和适用条件保存下来

Flash 的名称错位、抽屉的目标混淆和长任务的反复调整，都说明记忆除了操作顺序，还要保存适用条件与失败证据。例如“目标没有随夹爪抬升”可以直接记录；“接近高度一定错误”仍是待检验的解释。技能条目应写明对象绑定、坐标来源、前置条件、完成检查和退出条件，使新场景能够重新定位，而不是照搬旧坐标。〔L03、L10–L12；E3 综合分析〕

COAP 提供了把显式状态和操作分支写入程序的参考，正好对应 RPent 已观察到的重复工具往返；迁移时仍使用现有感知和 π。EmbodiedEvo 则提示如何组织经验：其开发集用 Skills／Lessons 时成功率为 30.17%，原始历史为 29.78%，对应每集 Test 费用为 5.732 与 12.203 美元；这是作者按固定模型费率

计算的测试成本，经验获取成本另列。对我们最有用的问题是，同源的结构化经验能否保留恢复判断，同时减少读入历史的成本。〔L11、L12、W11、W12；E3 作者结果与研究推断〕

因此可保留当前 RPent 记忆不变，分别增加诊断 Lessons、同源 Lessons 加已验证 Skills，以及同一次探索的完整公开历史。固定选择器与执行技能后，比较任务成功、错误经验引发的动作、输入量和整集时间；探索与验证成本另记。它承接前面的状态检查和恢复实验：先确认哪些做法有效，再把适用条件保存下来，供后续任务调用。〔L12；E3 设计〕

到这里，实验已经展示了三种额外工作：逐块审核、未推进任务的重复调用，以及错误后的恢复。接下来用 RPent 的整批计时和独立深测，计算这些工作实际消耗了多少时间，再判断先改哪一段。〔E3〕

4 RPent 性能 从整集差异到恢复过程

4.1 120 集的成功数接近 完成成本不同

性能分析先计算为了得到一个成功结果，系统实际消耗了多少时间和调用。RPent 正式对照覆盖 3 个套件、每套件 20 个相同任务身份、Astra 与 Sol 各 60 集，共 120 集。环境成功为 Astra 50/60、Sol 49/60，累计回合工作量为 123408.4 秒，记录到 5014 个独立计量响应。总体成功数仅差 1 集，但这不等于两种策略的运行效率接近。〔E3；S1 逐集复算〕

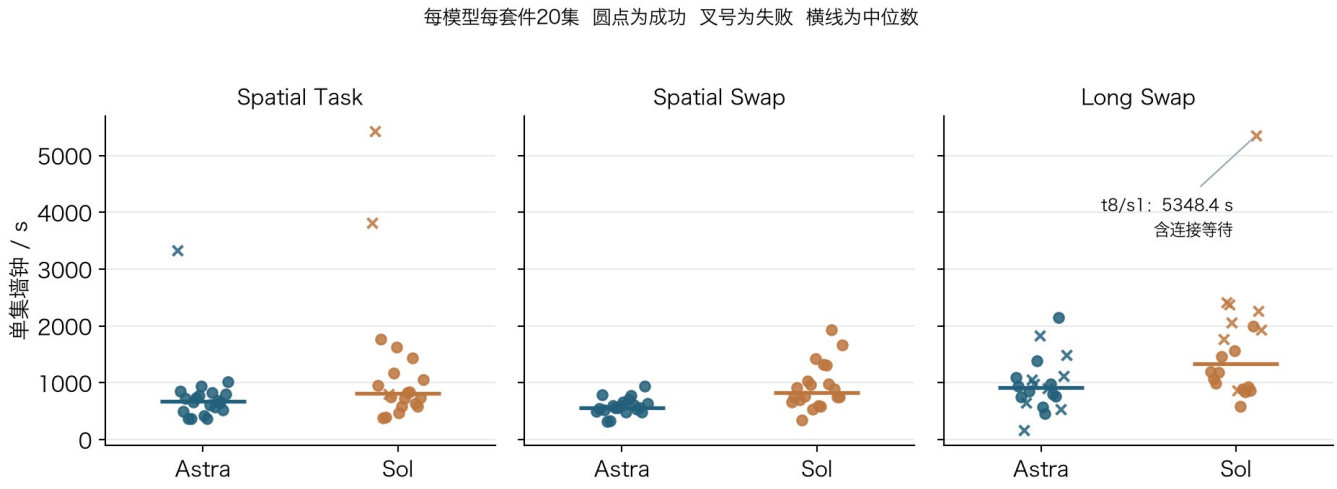


图 01 每点为一个回合 圆点成功 叉号失败 横线为中位数（S1 E3）

表 14

套件／模型	成功数	均值 s	p50 s	p95 s	平均请求数
Spatial Task／Astra	19/20	787.42	676.85	1134.41	32.00
Spatial Task／Sol	17/20	1248.62	811.60	3888.39	43.20
Spatial Swap／Astra	20/20	586.92	560.05	801.08	28.95
Spatial Swap／Sol	20/20	943.69	828.85	1674.90	43.25
Long Swap／Astra	11/20	973.72	919.45	1845.14	44.65
Long Swap／Sol	12/20	1630.06	1332.75	2565.85	58.65

同套件每模型 n=20，时间含成功与失败回合；p95 按排序后 (n−1)×0.95 线性插值。输入 Token 累计含每轮重复历史，cached input 是 input 子集，reasoning 是 output 子集。S1；统计〔E3〕。

三个套件里，Astra 的平均请求数和平均回合时间均较低。Spatial Task 的 Sol 均值为 1248.62 秒、p50 为 811.60 秒，p95 升至 3888.39 秒，说明少量长回合对总成本影响很大。只报均值会把这种尾部摊平；只报成功率又会丢掉成功之前反复尝试的代价。因此后续评测同时保留成功数、整集时间分布、调用数和失败恢复时间。〔E3〕

Long Swap 更能说明指标选择的重要性。Astra 全批 19474.4 秒得到 11 个成功产出，Sol 全批 32601.1 秒得到 12 个成功产出，分别相当于每个成功产出承担 1770.40 秒和 2716.76 秒本批工作量。这个“全批总耗时÷成功数”把失败代价计入产出成本，适合评估实验吞吐；成功回合均值则描述已经完成任务时的效率，两者需要同时阅读。〔E3〕

对研发而言，这两个指标对应两类改进。缩短成功路径能提高已经会做的任务的吞吐；减少失败路径的反复尝试，则既释放实验资源，也有机会提升成功产出。当前 Long Swap 的成功数相近、产出成本差距较大，因此下一阶段应把失败类型与恢复代价列为主要指标，并比较延长时间预算的收益。〔E3；研究判断〕

下一步必须把“同一任务完成得更快”与“成功、失败构成不同”分开，才能决定优化模型单次推理，还是优化动作选择与恢复。为此，先按套件、任务和 seed 配对，再进入每集时间的构成。

4.2 配对任务中的额外时间来自哪里

Spatial Swap 的 20 对任务均由两模型完成，是观察同完成条件下效率的主证据：Astra 累计 11738.4 秒，Sol 18873.7 秒；Astra 总耗时少 37.81%。相同任务身份的 Long Swap 若使用全部 20 对，Sol 比

Astra 多 67.40%；缩到共同成功的 8 对后，两者分别为 7934.2 与 8470.6 秒，差距变成 6.76%。因此，Long Swap 的全批差异显著受失败路径及长尾构成影响。共同成功子集用来解释完成后的效率，全批保留失败成本，两者共同支持判断。〔E3；S1〕

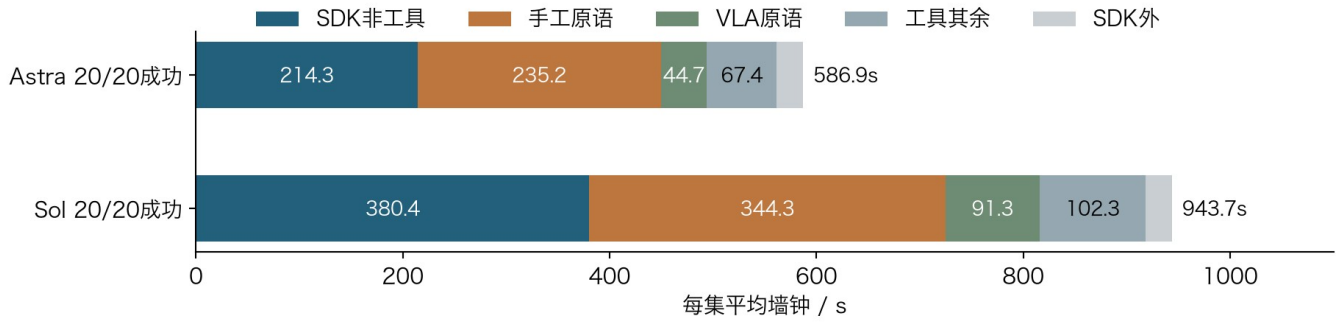


图 02 Spatial Swap 每模型 20 集均成功 五类平均时间相加等于平均回合时间 SDK 非工具含模型请求与相关等待〔S1 E3〕

表 15

互斥时间类别	120 集累计 s	占回合工作量	Spatial Swap 的 Sol−Astra s
手工原语	45327.570	36.73%	2182.320
VLA 原语	9367.080	7.59%	932.990
工具其余	12429.824	10.07%	697.927
SDK 非工具	53394.026	43.27%	3321.863
SDK 外	2889.900	2.34%	0.200

五类按互斥区间归并，总和等于 episode 工作量；恢复动作归入对应工具，模型加载另列。120 集模型加载累计 20817.6 秒，不加回本表。Spatial Swap 差额分母为 7135.3 秒。S1；统计〔E3〕。

全批 SDK 非工具占 43.27%，手工与 VLA 原语合计 44.32%，是两块接近的主要成本。SDK 非工具包括模型调用、运行时和通信等待；动作原语包含技能执行中的仿真、观测和渲染。五类回答“时间落在哪里”，还没有回答“落在某类里的哪个计算阶段”。将 VLA 原语全当作 π 模型前向，或将 SDK 非工具全当作 thinking，都会把优化目标选偏。〔E3〕

在完成率相同的 Spatial Swap 中，SDK 非工具解释总差额的 46.56%，手工原语与 VLA 原语解释 43.66%，工具其余解释 9.78%。Astra 的 579 次请求、186 个动作、175 个 π 块，对应 Sol 的 865 次请求、286 个动作、407 个 π 块。更长的运行伴随着更多调用和执行；性能差异已经跨过云端和本地两侧。〔E3〕

进一步把 SDK 非工具时间分成调用次数与每次均摊成本两个因素。Astra 每次分摊 7.401 秒，Sol 为 8.794 秒。用两模型平均成本计算新增 286 次调用的影响，再用两模型平均调用数计算单位成本变化，3321.863 秒差额可分成两项：调用数项 2315.970 秒，占 69.72%；均摊成本项 1005.893 秒，占 30.28%。这是一种对称的账面分解，均摊成本包含请求内容与服务等待的变化。它说明当前 SDK 差额中，调用数量占很大权重，后续需要检查这些调用完成了哪些有效判断。〔E3；公式和精确值见复算附件〕

Token 账本也呈现同一方向：Spatial Swap 的累计输入为 Astra 3350.19 万、Sol 5843.51 万，累计输出为 92009 与 161138 Token；Sol 的 reasoning Token 更达到 Astra 的 5.34 倍。输入反复携带历史，减少无效调用因而不仅节省固定往返，还会截断后续上下文增长。至于 reasoning Token 多是否代表更慢的思考，仍应以有效阶段时间检验；这正是后续引入 DeepSeek 流式分段的原因。〔E3〕

Long Swap 的 13126.7 秒总差额中，共同成功 8 对只贡献 536.4 秒，其余 12 对贡献 12590.3 秒，占 95.91%。这 12 对包含一方成功或双方失败的组合。它把后续轨迹审查的重点进一步缩小：优先定位任务在哪个阶段分叉、失败后怎样恢复，再检查已共同成功路径上的单次执行速度。〔E3〕

由此形成两个并行问题：执行侧到底在渲染、物理还是 π 上花时间；多出来的调用是否来自重复抓取和恢复。下一组独立深测正是为这两个问题增加采集粒度。

4.3 八集深测定位渲染成本和动作块延迟

深测选择 t0、t9 两场景，各两个 seed、两个模型，共 8 个采纳回合，包含 3 次成功和 5 次正常失败。累计工作量 10965.3 秒，448 个逻辑采样、473 次采样尝试、448 个完成响应，以及 371 个 π 动作块。它是独立实验，用来解释内部机制；正式 120 集的成功率分母保持原样。另有两个 OOM 尝试消耗 3205.2 秒回合工作量，单独计入基础设施成本。〔E3；S2〕

表 16

深测阶段	累计 s	占 8 集工作量／内部占比	说明
sim.render	4391.179	40.05%	原语窗口内渲染
物理 step1＋step2	121.481	1.11%	物理步进
π handler	198.653	1.81%	371 次模型服务处理
其中 π prefix	96.186	handler 的 48.42%	前缀处理
其中 denoise core	86.142	handler 的 43.36%	去噪核心

prefix、denoise 嵌套于 handler；渲染/物理嵌套于动作。各层级不能再次相加。S2 report-data、quantitative-conclusion；统计〔E3〕。

执行侧最明显的热点是图像生成与读回。源码显示相机 observation 通过 sim.render 生成，环境步进循环更新 observables，离屏渲染路径包含场景更新和像素读回。也就是说，动作执行会持续触发视觉观测成本，并非只有显式“看一眼”时才发生。优化应先测相机采样频率、分辨率、无消费图像的渲染次数和读回同步，而不是一开始就假设物理求解器最慢。〔E1；S3 render-path-audit 引用源码〕

若 π handler 耗时减半、其余工作量保持不变，整批预计节省 99.326 秒，即 0.91%。若只从账本扣除已测渲染，剩余 T* 为 6574.121 秒，此时同一项节省占 1.51%，SDK 非工具占 T* 的 63.70%。这是一项定位优先级的敏感性分析：仿真吞吐先查渲染；减少高层等待和重复调用，在扣除渲染后依然重要。〔E3；条件计算〕

实时控制的排序不同。固定观测校准采用相同 π 权重、输入与种子，关闭 mask 计数同步；预热后 none 与 stages 各测 3 次，中位完成延迟为 361.047 与 363.719 毫秒，插桩增加 2.672 毫秒，即 0.74%，动作逐字节相同。该输入一次返回 5 个环境动作。以 20 Hz 控制目标、每步 50 毫秒计算，5 步覆盖 250 毫秒，361 毫秒推理已超过动作窗口，需要约 1.44 倍加速，或靠提前推理与缓冲覆盖。〔E2 配置；E3 复算及条件计算；S3〕

加载与异常恢复也进入部署账本：120 集记录的加载累计为 20817.6 秒，深测中两个 OOM 尝试还额外消耗了 3205.2 秒回合工作量。常驻模型服务可以摊薄加载，但资源不足导致的重启会重新支付这笔成本。下一次吞吐测量应先报告冷启动、预热后正常回合和异常恢复三组时间，再比较常驻服务及并发策略。这样，芯片利用率提升才会体现在整批有效任务产出上。〔E3〕

所以“π 在整集里只占 1.81%”与“π 仍限制连续控制”并不矛盾：前者是累计吞吐占比，后者是局部交付期限。后续本地实验应同时记录整集工作量、动作块 p95/p99、buffer 耗尽与 deadline miss。接着再看轨迹，确认多少动作块实际上消耗在重复恢复上。

4.4 重复抓取怎样拉长请求与动作链

t0/seed1 给出一个能直接讲清楚的问题链：Astra 在 1049.1 秒完成，Sol 用了 2176.5 秒后失败。按客户端时钟分别画请求生命周期与工具区间，两者都呈现多次往返，Sol 的请求和动作链持续得更久。请求生命周期可能跨越工具区间，图中两行用于显示顺序和间隙；本地 π、渲染与动作时间使用独立时钟，阶段归属通过动作序列对应。〔E3；S2 timeline_v2〕

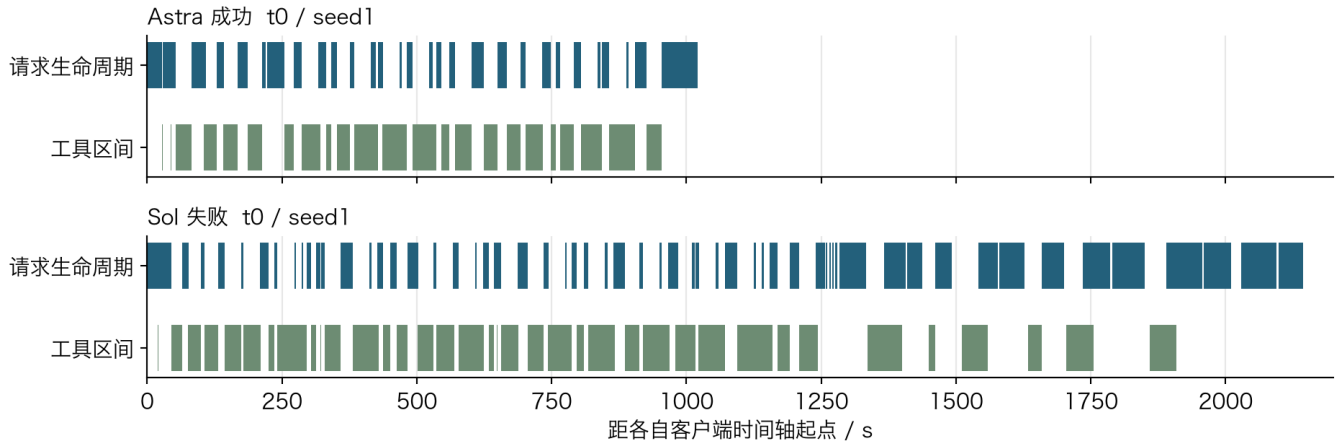


图 03 t0 seed1 客户端请求与工具时间轴 两行有重叠 不相加〔S2 E3〕

表 17

t0/seed1	回合 s	逻辑采样／尝试	π 块	SDK 非工具 s	渲染 s
Astra 成功	1049.1	41／42	15	415.093	389.224
Sol 失败	2176.5	77／83	116	1009.621	741.085

同场景、同 seed 的代表回合；用于机制案例，分母不是 120 集总体。请求时间轴仅用 client lane，工具/请求行不相加。S2；统计〔E3〕。

Astra 的 21 个本地动作中调用两次 pi0_pick，最后 move_to 后观察到任务终止；Sol 的 34 个动作中调用八次 pi0_pick，任务仍未完成。其逻辑采样增至 77、π 块增至 116，反映的不是单个推理阶段普遍变慢，而是抓取—观察—调整—再抓取链条被反复执行。每次重复既会增加模型输入历史，也会增加本地动作、相机渲染和再次验证。〔E3〕

Sol 第 18–34 个动作累计执行 562.942 秒，其中渲染 447.543 秒、其余执行 115.399 秒；这些动作在本地时间轴上从 660.672 秒延伸至 1858.561 秒，跨度 1197.889 秒，中间还包含请求与其他等待。前一个数字度量恢复动作自身成本，后一个数字体现恢复链经过多少墙钟时间。二者合起来解释了为什么仅加速 π 前向无法消除整个长尾。〔E3〕

这里最直接的 Harness 改进是把技能前置条件和完成条件显式化：抓取后检验物体是否随夹爪运动，失败后先重定位并限制无进展重试，再选择局部修正或高层重规划。下一轮保持 planner 与 π 权重不变，先加入状态验收，再加入恢复策略，比较每个有效抓取的请求数、π 块数、恢复段时间和终局成功。这样能把“模型判断得更准”转化为可归因的少调用、少动作。DeepSeek 的思考档位实验进一步检验：让一次请求更便宜，会不会改变这条轨迹？〔E3；实验设计〕

5 DeepSeek 性能 从思考档位到请求阶段

5.1 降低思考预算后 单次与整集可能反向变化

DeepSeek 两场景实验分为原始非流式 8 集和独立流式补测 8 集，各含 medium、none 两档。每批每档都是 t0 两身份记录 recipe/finish 成功，t9 两身份未成功；因此下表用两档共同成功的 t0 做配对，并将 t9 的失败/退出方式保留在附件。成功依据采用该批 finish 记录，后续统一接入环境终态评估器。〔E3；S4 audited-data〕

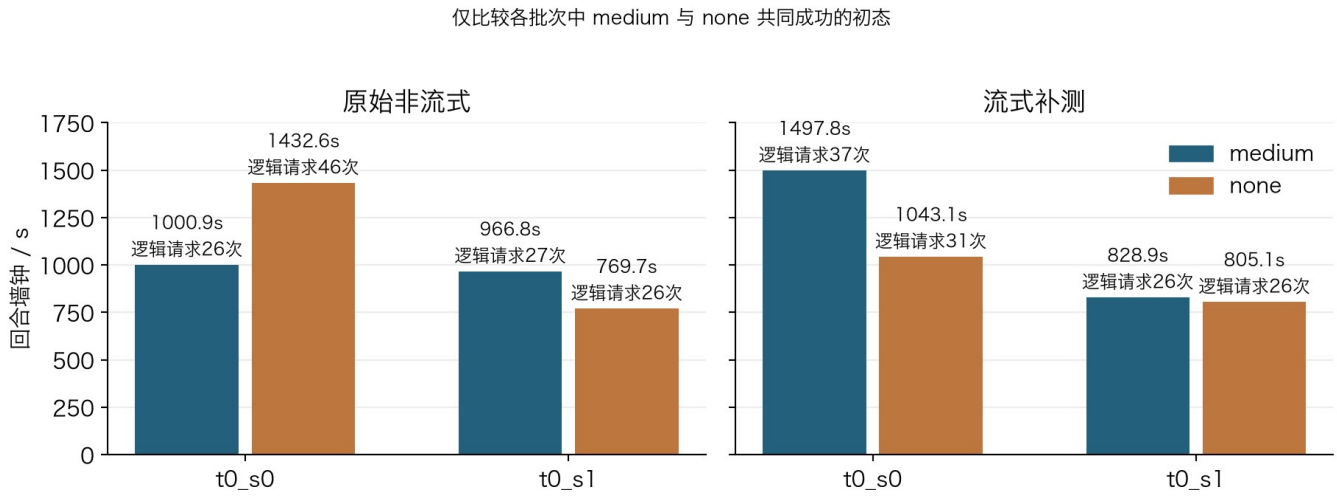


图 04 两批分别配对 标签保留逻辑请求数〔S4 E3〕

表 18

批次／身份	medium→none 回合 s	none 时间变化	逻辑请求	工具调用
非流式 t0/s0	1000.9→1432.6	＋43.13%	26→46	46→102
非流式 t0/s1	966.8→769.7	−20.39%	27→26	78→51
流式 t0/s0	1497.8→1043.1	−30.36%	37→31	80→62
流式 t0/s1	828.9→805.1	−2.87%	26→26	63→52

每行同批同任务同 seed 配对；批次间服务时段、轨迹和输入均会变化。8 组流式初始工具数均为 17、输出上限 8192，但 system/tools 内容哈希随运行不同。下轮去掉动态运行字段后冻结语义内容。S4、S5；统计〔E3〕。

非流式 t0/s0 是“单次更快但整集更慢”的明确案例：none 的累计请求时长÷逻辑请求数为 17.514 秒，medium 为 18.391 秒，均摊值下降 4.77%；然而请求数由 26 增至 46、工具数由 46 增至 102，累计请求时间从 478.177 增至 805.664 秒，整集最终多花 431.7 秒。降低思考预算带来的单次收益，被更长的执行路径覆盖。〔E3〕

两份非流式 t0 合计，none 从 1967.7 增至 2202.3 秒，多 11.92%；两份流式 t0 合计则由 2326.7 降至 1848.2 秒，少 20.57%。流式 t0/s0 同时减少调用和请求均摊成本，因而收益最大；流式 t0/s1 两档逻辑请求均为 26，累计模型请求节省 94.495 秒，但整集只节省 23.8 秒，其余回合成本相应增加 70.695 秒。可见动作路线与工具耗时即使在请求数相同的时候，也会抵消云端收益。〔E3〕

这组证据支持把 low/none 设为任务阶段相关的预算选择：标准化状态确认可优先低预算，空间歧义、失败恢复和子目标切换保留较强判断。验证分两层展开：先用同请求体测延迟和输出一致性，再用同初态跑闭环，观察成功率、调用数与恢复成本。为了知道单次请求到底该改哪里，下一节进一步拆开首输出等待与输出生成窗口。〔E3；实验设计〕

5.2 流式配对任务主要等待首个有效输出

计时采集从响应流中寻找首个非空 reasoning、content 或 tool_calls delta，最早出现者定义为有效首输出。TPOT 用“首有效输出至最后 chunk 的时长÷(completion tokens−1)”计算，是每个请求的均摊输出时间。它包含流传输与协议尾部；可见 thinking 为首 reasoning 至首正文/工具输出的窗口。完整请求、TTFT、窗口和 Token 分配估算分别保留，服务端 prefill 与 kernel 时长由后续服务端 trace 补齐。〔E1；S5 measurement-source、proxy-source-audit〕

原始非流式批次有 306 条响应记录，缺有效首输出标记；流式补测有 178 次尝试、177 个逻辑请求、177 个有效 TTFT。单独采集的代理库存另有 25 文件、484 行：其中 355 条有效流式记录为 DeepSeek 287 条、Qwen 68 条。经过运行映射，最终阶段表由流式 178 行和精确归属的 DeepSeek 280 行组成，共 458 行、23 组，其中 457 行有有效 TTFT；v5 等单局已经包含其中。这个身份台账保证后续详表复用数据时不重复增加实验数。〔E3；S4〕

表 19

流式共同成功 t0	medium	none
逻辑请求／尝试／有效 TTFT	63／64／63	57／57／57
完整请求 p50／p95，s	17.371／40.855	12.272／21.290
TTFT p50／p95，s	11.891／21.359	10.595／16.893
TPOT p50／p95，ms/token	5.667／14.171	4.374／7.105
累计完整请求，s	1238.435	715.819
累计有效 TTFT，s	776.094	607.904
累计可见 thinking／正文工具窗口，s	376.252／73.677	0／107.915
失败尝试，s	12.412	0

每档 2 个成功身份，分位数在逐请求记录上计算；medium 的失败尝试纳入完整请求总量，TTFT 与窗口仅纳入有效行。TPOT 的 p50/p95 是请求均摊值的分布。S4；统计〔E3〕。

这四集累计请求 1954.255 秒，其中有效 TTFT 为 1383.998 秒，占 70.82%。none 去除了可见 thinking，仍需等待 607.904 秒才陆续拿到首输出。因此该批优先事项是减少输入准备、传输、历史上下文与服务等待，再进一步压缩输出。若只把 thinking 清零，剩余的首输出等待依然构成主要成本。〔E3〕

输出组成也发生了变化。medium 的 61835 个输出 Token 中，45618 个是 reasoning，余下 16217 个为其他输出；none 共输出 23370 个 Token，比 medium 的非 reasoning 部分多 44.11%。尽管 none 少了 6 个逻辑请求，正文/工具窗口仍从 73.677 增至 107.915 秒。这说明关闭思考后，输出协议、参数长度和调用内容也会改变；优化完整请求应同时限制重复解释和无必要的工具参数，而不是仅把 reasoning Token 当作全部可削减项。〔E3〕

medium 的可见 thinking 累计 376.252 秒，按 reasoning Token 占比平分输出窗口得到的估算为 352.550 秒，两者相差 23.702 秒。这不是重复测了同一段时间，而是事件窗口和按 Token 分摊两种定义。正文用观测窗口定位问题，估算列留给预算分析；同样，旧报告的 0.7 毫秒“TPOT”实际是 delta 字段到达间隔，不能与本表按 Token 归一的值合并。〔E1 定义；E3 复算〕

首输出占比由输入形态和输出任务共同决定。代理阶段表中的后续 v5 单局恰好提供另一种情形：首输出已经较短，长尾转移到了思考和工具参数输出。下一节沿它的 15 次请求逐条展开。

5.3 v5 的两次长请求分别卡在思考和记录输出

v5 单局记录 15 次请求、182.5 秒回合时间。请求累计 93.592 秒，均值 6.239 秒，而 p50 只有 3.548 秒，p95 达 18.924 秒。首输出累计 27.256 秒占 29.12%，首输出之后 66.336 秒占 70.88%，与上一组共同成功任务的瓶颈位置相反。图 05 保留全部 15 次请求，按 TTFT、可见 thinking 和正文/工具窗口分段。其中第 11 次和第 14 次的等待最突出。〔E3；S4、S6〕

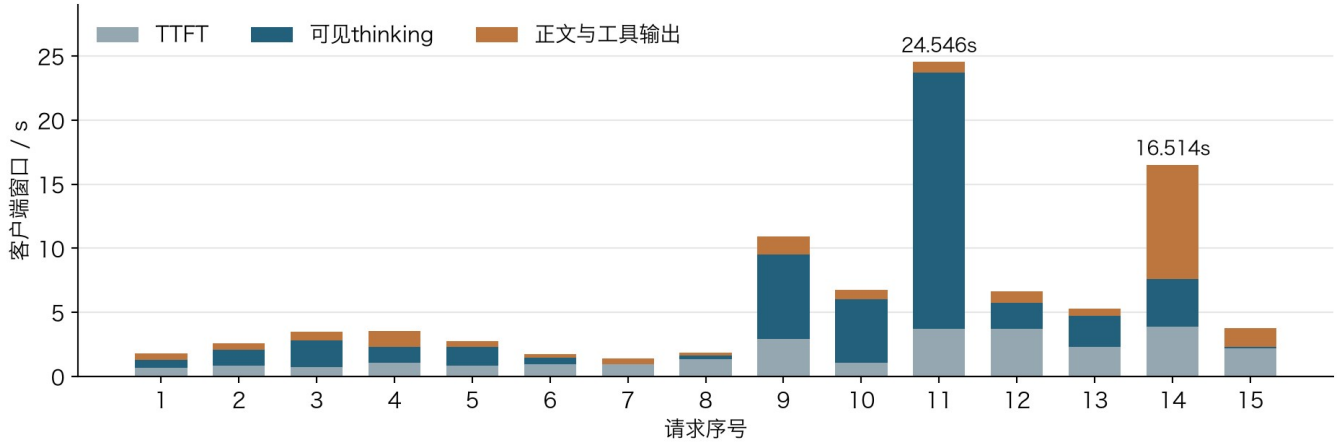


图 05 v5 全部 15 请求的有效事件窗口 第 11 和第 14 次合计占请求时间 43.87%〔S4 S6 E3〕

表 20

代表请求	完整 s	TTFT s	可见 thinking s	正文／工具 s	输出特征
第 11 次	24.546	3.731	19.959	0.856	4491 reasoning Token
第 14 次	16.514	3.886	3.689	8.939	2687 其他输出 Token；两次 write_text_file

使用 v5 同局后续代理审计的逐请求事件窗口。15 次含初始化、读取、审计及收尾，不等同 15 次机器人动作。S4 streaming-phases、S6 逐轮工具表；统计〔E3〕。

这两次请求合计 41.060 秒，占全局请求时间 43.87%。第 11 次主要花在长思考，第 14 次主要花在收尾记录输出；对两者采取同一种“模型加速”策略会错过便宜而直接的改法。前者适合比较思考预算、候选动作是否过多及是否存在反复自检；后者应缩短记录 Schema、去掉重复描述，并在动作完成后异步写入非决策信息。〔E3〕

v5 全局可见 thinking 为 47.144 秒，正文/工具窗口为 19.192 秒。其 TTFT 可分为代理前处理 9.930 秒、请求至响应头 10.415 秒、响应头至首输出 6.907 秒，独立舍入使三项与 TTFT 总量相差 4 毫秒。前处理包括图像处理及文件接口操作；响应头前等待包含发送、链路与服务端接收/排队。现有时间轴能清楚展示可操作的客户端阶段，后续服务端 trace 再把其中的排队与计算分开。〔E1 定义；E3 复算〕

为了安排实验优先级，可以先对具体长尾做预算测算：仅将第 11 次的 thinking 窗口减半，假定输出动作和后续路径保持原样，可节省 9.980 秒，相当于该局请求总量的 10.66%、回合时间的 5.47%。这比将所有轮次统一乘一个加速比更贴近当前负载。第 14 次则先检验缩短记录是否减少 8.939 秒的正文/工具窗口，并确认记录并不参与后续决策，再决定移出关键路径。〔E3；条件计算与实验设计〕

另一单局 ds_t2_par 同为 15 请求，但回合为 205.3 秒、请求累计 110.534 秒，TTFT 21.739 秒，首输出后 88.795 秒，占 80.33%。相同的请求数量没有带来相同请求成本，说明输出内容和各轮长度仍然重要。两个单局共同指向“把长思考与长工具输出分别治理”；它们保留各自身份，作为两种具体工作负载，不构成某项改动的前后收益。〔E3〕

接下来的输入实验则要回答另一个问题：v5 为什么可以在保留更多历史图像的同时让请求体变小，字节、上下文 Token 和缓存分别发生了什么变化?

5.4 分别检查传输字节 上下文和缓存

v5 对同一请求进行图像文件化与引用替换，平均传输载荷由 3136.1 KB 缩至 185.7 KB，末轮由 10118 KB 缩至 280 KB；但上下文仍从 13094 增长到 84686 Token。缩小序列化字节并不等于模型处理的信息历史同步缩短。网络载荷、模型上下文和缓存分别描述传输量、历史增长与重复输入复用。图 06 展示了各轮上下文、缓存和首输出等待的变化。〔E3；S6〕

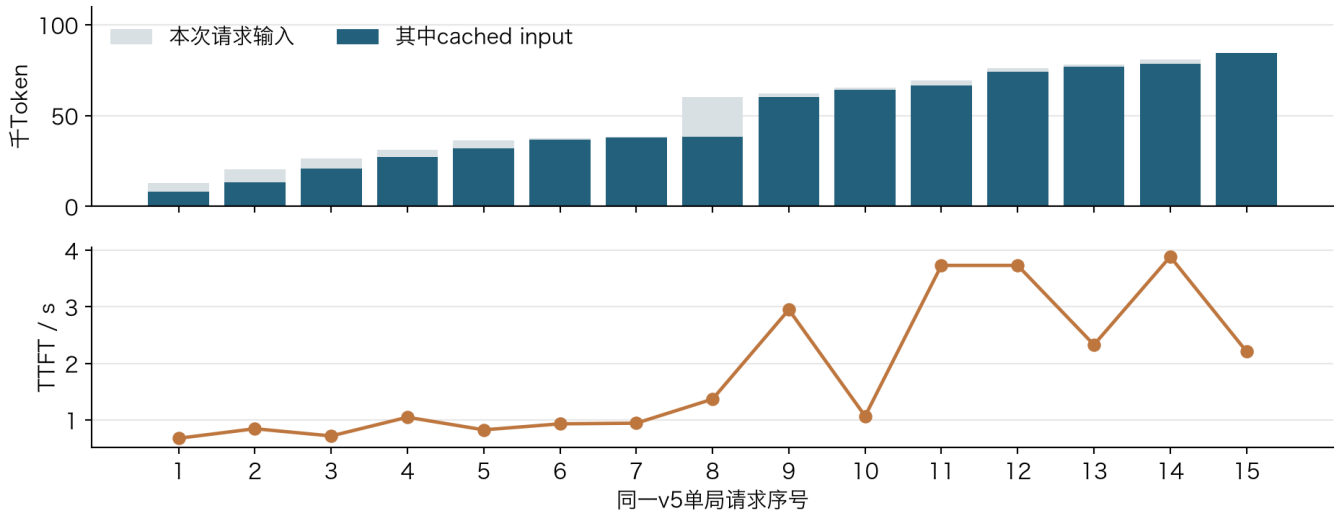


图 06 v5 各轮输入与缓存 Token 及有效 TTFT 缓存是输入子集〔S4 E3〕

表 21

单局身份	请求数／回合 s	累计请求 s	累计 TTFT s	输入与缓存／输出
v5	15／182.5	93.592	27.256	input 780723；cached 720256；output 17432
Files API 独立局	11／153.1	100.5	57.762	input 500355；output 10722

v5 使用后续映射成功的阶段表；Files API 独立 11 请求局按其原报告逐轮表复算。Files API 的 thinking/decode 为 Token×速度估算。两个单局分别保留，不作为版本前后差额。S4、S6；统计〔E3〕。

v5 加权缓存率为 720256÷780723=92.25%，表示累计输入中有很大部分被复用；历史报告的 88.5%是逐轮缓存比例的非加权平均，两者权重不同。高命中率和很长的上下文可以同时存在，服务仍需处理新增图像/文本、输出与调度。因此缓存命中率是解释变量，最终验收仍是完整决策延迟和完成任务所需成本。〔E3〕

该局累计未缓存输入仍有 60467 Token。对输入优化，更有意义的比较是保持状态信息完整时，每轮新增和未缓存部分能否减少；对缓存优化，则关注稳定前缀是否被动态字段或图像裁剪破坏。图像传输去重、历史摘要和稳定前缀是三项不同改动，应逐项测量，避免一个高缓存百分比掩盖仍在增长的总输入量。〔E3；实验设计〕

Files API 独立局的请求均值为 9.136 秒、p50 为 6.8 秒；TTFT 均值 5.251 秒、p50 为 3.649 秒。原表 thinking 与 decode 估算分别累计 23.5 和 19.3 秒，概览使用的 3 ms/token 与逐轮约 4 ms/token 假设另行标注。其首输出比 v5 更长，说明“用了 Files API”本身不足以决定 TTFT：输入规模、缓存状态、服务时段和输出轨迹共同变化。下一次应在同一请求体上只切换传输方式，再比较完整链路。〔E3〕

具体做法是固定任务状态、图像内容、提示、工具与输出 Schema，交替发送内嵌图像和文件引用两组；记录首次上传、重复引用命中、请求 bytes、未缓存 Token、有效 TTFT 与最终动作参数。历史压缩单独开一组，检查是否保留了抓持状态、失败尝试和子目标进度。这样能分别回答“传输少了多少”和“模型是否仍得到完成决策所需信息”。已有的同请求体重放与上传阶梯为这类控制变量实验提供了起点。〔E3；实验设计〕

5.5 固定请求重放与载荷阶梯测到了什么

保存的安全重放指标覆盖 9 份实际请求体、42 次发送，其中 33 次流式、9 次非流式，均无记录错误。相同请求体的重复发送比跨机器人轨迹比较更接近控制变量，但缓存状态仍随请求改变。下面选四个流式组展示输入量级与等待分布，完整 9 组留在附件。〔E3；S7 stream-probe〕

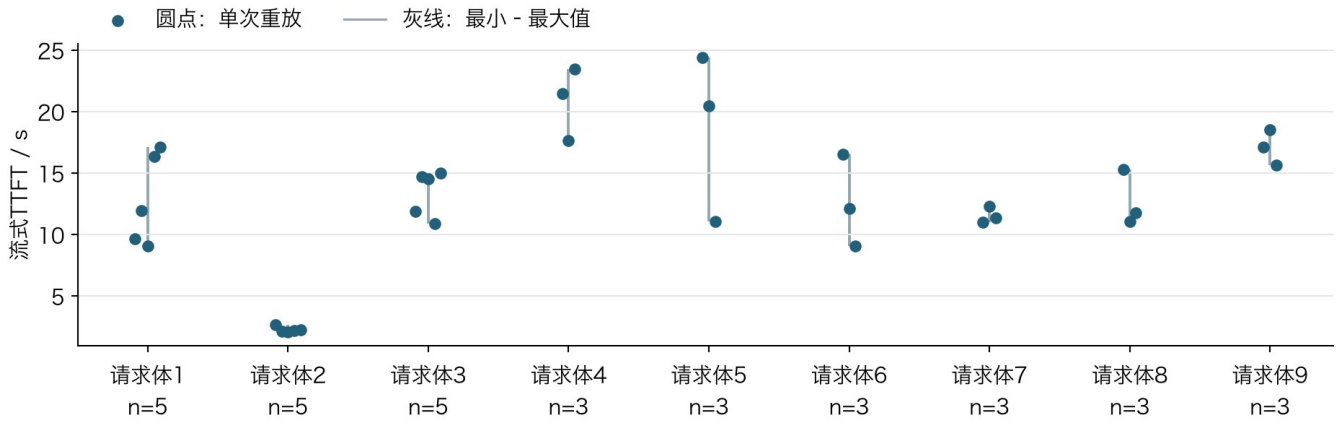


图 07 9 个请求体的 33 次流式重放 每点为一次 TTFT 其余 9 次为非流式〔S7 E3〕

表 22

请求体组	输入 Token	流式次数	TTFT p50 s	TTFT 范围 s
t0/s0 medium 小体	13372	5	2.176	2.052–2.671
t0/s0 medium 大体	64652	5	11.974	9.070–17.138
t0/s1 medium 大体	81748	5	14.511	10.861–14.971
t9/s0 medium 大体	229734	3	21.490	17.648–23.447

每组固定请求体哈希；组间内容和 Token 长度不同。同组缓存 Token 可变，例如 64652 输入组记录到 0、128、64512。分位数仅描述该组少量重复。S7；统计〔E3〕。

这些结果说明两个层次：较大的实际输入伴随更高首输出等待；同一 64652 Token 请求体仍可在 9.070–17.138 秒之间波动，固定输入后服务与缓存条件的影响依然显著。因此下一轮 A/B 需要按冷/暖缓存分层

并交替发送，记录服务时间窗；只把首次冷请求与随后热请求作前后比较，会把缓存预热记到代码优化上。〔E3〕

载荷阶梯进一步把字节成本单独暴露出来。约 0.05、0.5、1.5、4.3 MiB 的请求，最短响应分别为 1.386、2.369、2.445、3.855 秒；最大体相对最小体多 2.470 秒，对应差分约 1.804 MB/s。测量采用返回 404 的网关路径，数字描述链路与接收处理，不包含正常模型推理。它的价值是证明大载荷本身有可观时间成本，从而支持减少重复图片传输，并为真实请求增加独立的发送/接收观测。〔E3；S7 upload-ladder〕

历史输入阶梯中，流式中位拟合斜率为 0.064722 ms/token，非流式为 0.019076 ms/token；若改用各点最小值拟合，两者为 0.065615 和 0.066079 ms/token，更为接近。输出阶梯中位拟合斜率为 1.391546 ms/token。换汇总方式就会改变系数，是服务波动影响回归的直接信号；这些参数适合该网关的经验预算，精确 prefill/decode 归因应由服务端排队时间、batch、KV 命中与 kernel trace 支撑。〔E3；S7 calibration〕

另一个双请求体的 stream/plain 验证共 16 次，结束原因均为 tool_calls，但输出长度及工具调用数会变化。传输形式的切换还应验收工具参数和执行结果的一致性，不能只看接口是否返回成功。由此，下轮实验的控制对象应从“模型名和任务名相同”推进到“输入内容、缓存状态、输出协议与验收结果均可对齐”。〔E3；S7 cliff-ab〕

6 下一轮改进 对应已有结果设计实验

6.1 把训练重点放到抓持确认和失败后的修正

前两章的计时把改进方向收窄了：调用数的差异需要由状态判断和 Harness 解释，首输出等待需要从输入与服务链路处理，连续执行还要求动作块按时返回。下面按模型训练、系统机制和运行调度分别设计实验，再讨论联合架构。〔E3〕

前面的实验把训练目标具体化了。早期抓放的 60 次任务中，有 48 次停在稳定抓取之前；Astra 的恢复案例则显示，观察到方块没有随手抬升后，重新调整抓取位置能让任务继续。下一批数据应重点覆盖接近、闭合、试举这几个连续状态，以及抓空之后的重新定位。把整条成功轨迹平均切成片段，会让大量平稳移动淹没这些真正决定结果的时刻。〔L02，E3〕

我们建议先训练状态判断，再训练动作选择。第一步让模型辨认目标对象、可放置区域、是否抓住、是否放稳和当前任务阶段；第二步提供几个可执行候选，让模型选择继续、重观测、重新接近或切换技能。这

样，位置估计偏差与决策选择错误可以分别检查，后续的 JEV 对照也能沿用同一套候选。〔E3，实验设计〕

表 23

已有实验中的现象	应补的样本和标签	训练后的检查项
抓空后继续抬升或搬运	闭合前后及试举图像；是否随手移动的标签	假抓持率与抓空后的恢复成功率
底层抽屉任务打开了中层	目标部件、相邻部件、动作后关节状态	目标部件识别与阶段完成判断
反复调用 pick 但没有推进	重复动作前后的观测；候选恢复及执行结果	无进展重复次数与每次恢复耗时
Astra 修正位置后重新抓取	教师选择、实际动作、下一观测与任务结果	同状态下的选择正确率与闭环成功

表中现象来自 L02、L03、L06；训练与评价设计为 E3。仿真状态用于离线标签和独立评分，在线策略保持既定观测接口。

蒸馏时保留教师选了什么以及执行后发生了什么。教师解释可作为辅助信息，主监督由实际后果确认。随后让学生自己运行，收集它容易偏离的状态，再补充教师纠正或短分支试验结果。这样增加的是学生真正会遇到的难例。DAgger 的状态补标思路与此一致；RoboSpatial、SpatialVLM 提供空间关系监督的参考，具身后训练工作则可用于设计定位和纠正任务。〔W05–W10，E3〕

SFT 和蒸馏之后，再用任务回报优化“何时继续、何时恢复、何时结束”。第一轮固定执行器与 Harness，分别比较基座、状态 SFT、加入决策蒸馏和加入 RL。既看终局成功，也看假完成、错误抓持判断和恢复耗时。若状态误判减少，下一步再比较固定复查与事件触发，确认减少观察后是否仍能及时发现抓空和滑落；若恢复成功率没有提高，则继续分析恢复候选和执行结果，补充相应难例。〔E3，实验设计〕

表 24

顺序	新增内容	要证明的增量
A → B	原基座 → 空间与状态 SFT	定位、抓持判断是否更准确
B → C	加入验证与恢复蒸馏	错误出现后是否更容易完成任务
C → D	加入学生访问状态的补标	能否纠正自己造成的偏离
D → E	加入高层 RL	恢复策略能否迁移到未见初态
E → F	成功率约束下压缩输出与调用	质量保持时，回合耗时是否降低

待实施训练消融〔E3〕。固定评测栈，按场景和轨迹划分数据；同时保留成功数、假抓持/假完成率、恢复成功率、调用数、输出 Token、请求及回合 p50/p95。

6.2 先确定哪些调用可以交给本地执行

Harness 的实验顺序沿用第三章：先校准阶段检查，再依次加入局部恢复、复合技能和事件触发。每一步同时比较新增成功、减少的高层调用和新增的本地计算。待这些机制稳定后，再固定状态与候选，更换规则、Astra 和 JEV 选择器；经验复用另设对照，避免同时改动执行机制和记忆内容。〔L09、L11、L12，E3〕

每项改动都保留一张实验结果表：同一任务与 seed 下，列出终局结果、阶段误报和漏报、逻辑请求数、工具与 π 块数、恢复时间及完整回合时间。若调用减少但失败增加，先检查漏掉了哪次必要观察；若质量保持而本地成本上升，则计算新增检查是否抵消了云端节省。这张表直接用于决定下一版本保留哪些机制。〔E3，实验设计〕

6.3 用动作覆盖时间选择协作频率

RPent 的累计时间说明渲染影响仿真吞吐，π 的单块延迟则影响下一段动作能否及时接上。两项指标需要分别优化。用于批量评测时，可先降低不必要的渲染、减少图像读回；面向连续控制时，要把完整动作块交付时间与当前缓冲剩余时间比较。〔L06，E3〕

用 20Hz 作为目标墙钟控制频率，每一步是 50ms。执行 5 步动作只提供 250ms 覆盖时间，而固定观测 π 校准的中位延迟是 361.047ms。即使暂不计额外调度，5 步方案仍相差 111.047ms，需约 1.44 倍速度或提前开始下一次推理。增加动作块长度也能扩展时间预算，代价是更久以后才利用新观测修正动作。〔L03、L06，E3，条件预算〕

表 25

每次执行步数 K	20Hz 下覆盖时间	动作块消耗速率	减去 361.047ms 的中位预算
2	100ms	10 块/s	−261.047ms
5	250ms	4 块/s	−111.047ms
8	400ms	2.5 块/s	38.953ms
10	500ms	2 块/s	138.953ms

预算按目标墙钟 20Hz 计算；已有 20Hz 是仿真控制协议，实际每墙钟秒推进步数另测。361.047ms 为固定观测微测中位数，各配置 3 次。K 取值限模型预测长度。〔E2/E3〕

上表用于挑选实验范围。正式调度用完整返回的 p95 延迟，再留出传输与调度余量；剩余动作必须覆盖这段时间。还要记录观测年龄，即动作提交时距离生成该观测过了多久。同样 400ms，在平稳搬运和接触对齐阶段可能带来不同结果，所以动作块长度需要分阶段测试。〔E3，实验设计〕

第一轮固定 VLA 权重，扫描 K=2、5、8、10，并比较同步与提前推理两种执行方式。每种配置记录成功、停顿、动作衔接、p95/p99 完整返回延迟和观测年龄。在相同调用预算下，再比较高层固定降频与事件触发。0.2、0.5、1Hz 可用于本地或延迟注入对照，云端候选频率先按实际完整决策延迟筛选。最终采用能维持任务质量且停顿较少的组合。〔L13、W13–W15，E3〕

异步执行能覆盖的等待，取决于当前技能有多长一段可以独立完成。服务端尚未返回时，机器人执行已确认的动作；回复到达后先检查任务阶段和对象状态，过期结果退出队列。RTC 的动作前缀约束可用于衔接低层动作块，高层则需要保留阶段版本。这样测到的收益才对应少了多少实际停顿。〔W13、L13，E3〕

6.4 分别优化思考预算和请求链路

导师最新实验报告 thinking 由 4.6 秒降到 1.4 秒，阶段减少 3.2 秒，降幅 69.6%。这条结果提供了值得复测的配置。结合我们已有 DeepSeek 实验，下一轮需要同时记录思考窗口、完整决策时间和动作结果；当请求数上升时，节省的单次时间可能被新的调用抵消。〔导师反馈 E4；L07 对照分析 E3〕

复测分两步。先冻结输入图像、system、tools、输出 Schema 及缓存条件，用交错时间顺序运行 low 和 medium，比较完整返回、TTFT、可见 thinking、输出长度和正确工具调用。再将相同配置放回固定初态，记录请求数、重复动作和终局成功。单次与整集结果各有一张表，就能判断收益来自计算预算，还是同时改变了执行路线。〔E3，实验设计〕

导师反馈的 4.6→1.4 秒描述思考阶段；已有独立校准中，medium/low 完整请求中位数为 8.949/7.463 秒，属于另一组请求。若思考原占完整请求比例 f，其他阶段与调用数保持不变，69.6% 的思考降幅对应完整请求约 69.6%×f 的降幅；例如 f=30% 时为 20.9%。实际验收仍以配对请求和闭环回合为准。〔S7，E3；条件计算 E3〕

表 26

已有数据指出的耗时位置	首个改动	保持固定的条件	验证的数据
共同成功四集 TTFT 占请求时间 70.82%	前处理、载荷、缓存或服务排队逐项测试	同请求内容、同输出方式与缓存分组	准备、上传、排队、服务端输入计算、完整返回
v5 思考 47.144 秒及第 11 次思考长尾	调整思考档位和输出要求	同一请求内容、交错运行顺序与固定工具 Schema	思考窗口、Token、选择正确率
v5 第 14 次正文工具输出 8.939 秒	精简记录和结构化输出	同一工具语义与字段	输出 Token、TPOT、完整结果可用时刻
π 单块中位 361.047ms	分 prefix 与 denoise 定位本地热点	固定图像、状态、动作长度和精度	设备 trace、完整块延迟、动作一致性

历史数据为 E3 复算，改动与固定条件为 E3 设计。各行分别立项比较，再把有效改动放回机器人任务。

对可本地部署的 VLM，使用固定请求和设备 trace 拆分视觉编码、prefill、decode、KV 读写及同步等待；π 则继续拆分 prefix 与 denoise。云端 API 按客户端已采到的阶段继续分析，服务端排队与输入计算在获得相应遥测后单列。每次优化处理一个已确认的热点，同时检查输出质量和完整返回时间。〔L06、L13，E3〕

优先级也随目标变化。批量仿真先处理 40.05%的渲染工作量和重复恢复；低延迟控制先满足动作块截止时间；云端 VLM 请求再按首输出前等待与输出阶段分组优化。这三组工作共享数据记录，但各自有清楚的主指标。〔L05–L07，E3〕

6.5 按失败类型组合 VLM JEV VLA 和 WAM

现有实验记录了目标混淆、抓持误判、重复选择无进展技能，以及接触和放置失败。前三类问题分别对应状态判断、技能选择与执行检查；抓空和放置偏差还提供了一组可用于检验预测模块的难例。融合架构先围绕这些已记录的问题分工，再逐项测量新增模块的收益与成本。〔L02–L09，E3〕

表 27

模块	对应已有实验中的问题	在首轮系统中的工作	单独评价
VLM	目标部件与任务阶段判断	根据观测生成状态、子目标和异常说明	定位、状态与子目标正确率
JEV 型选择器	重复选择无进展技能	从共同候选中选择继续、重定位或恢复	候选覆盖、选择正确率、选择延迟
VLA 及解析控制器	抓取、接触、移动和释放	按子目标产生并执行连续动作	技能成功、动作块延迟、衔接稳定
WAM 与评价模块	候选动作的后果难以判断	预测动作条件下的状态变化，再评价候选	预测误差、选对动作的增益、额外耗时

模块安排为 E3 设计。WAM 可联合建模视频与动作；预测与价值评价分别记录，评价由相应头或 evaluator 接口提供。

首轮由 VLM 确定子目标，规则或 JEV 选择技能，VLA 执行，WAM 在旁路预测所选动作的结果。先比较预测与真实下一状态；再在可重置的同一状态下执行不同候选，检查预测排序是否与实际效果一致。这项对照显示选择质量提高后，再将预测结果用于在线决策，并记录它新增的等待。〔E3，实验设计〕

这也解释了 low think、JEV 和分层 VLA 的关系。low think 调整一次模型调用的思考预算；JEV 替换有限候选的选择环节；分层 VLA 把连续执行交给低层，改变高层需要介入的次数。三者可以组合，但先分别测清正确率、调用数和完整耗时，再决定采用哪些部分。〔L03、L07、L09，E3〕

6.6 用 Motus 工程结果支撑联合后训练实验

模块化实验用于查明各部分的作用，共享表示的联合训练则研究这些能力能否在一个模型内部相互帮助。本周 Motus 工作已经提供工程起点：完整预训练基座约 80.6 亿参数，43,502,271 个适配参数参与双卡 FP32 联合更新；连续 10 步以及从 step9 恢复重做 step10 的检查均通过。〔L16，E3 转引〕

表 28

工程项	已完成结果
参与更新的部分	VLM 语言 LoRA、WAN LoRA、Action 与 Understanding 适配器、新 IO
数据与模型路径	完整预训练权重加载；真实冻结 VAE/T5 输入；两路图像进入 VLM forward
更新与恢复	双卡 FP32 连续 10 步；恢复后训练参数与 loss 一致
每卡显存峰值	allocated 约 57.34GiB；reserved 约 59.47GiB
运行环境	CANN8.5.2；torch_npu2.7.1.post2

来源 L16，2026 年 10 月 16 日工程记录〔E3〕。输入切分、BF16 数值与正式任务评估按后续记录推进。

在这个工程版本上，先完成数据时间对齐、初态隔离和监督暖启动，再进入任务 RL。对照保留四个版本：A 全部冻结并使用相同随机采样器；B 更新动作与理解连接；C 加入 VLM 内部 LoRA；D 再加入 WAN LoRA。各版本沿用相同起点、环境交互预算和评测初态。这样 B 到 C 回答任务反馈进入 VLM 是否有帮助，C 到 D 回答更新预测部分是否进一步改善控制。〔L14，E3 设计〕

训练时同时检查任务结果和预测质量。控制侧记录完成、恢复、动作数量与延迟；预测侧在固定动作条件下评价未来状态，避免各模型自己选择不同动作后难以比较。梯度记录用于确认任务回报实际更新了哪些模块，冻结和 detach 对照帮助定位贡献。连续动作生成的概率处理沿用预注册方案，工程实现与任务效果分别保留结果表。〔L14、W16、W17、W19，E3〕

下一轮交付应与前面每项发现对应：抓持和阶段错误对应状态训练结果，逐块审核对应触发与复合技能对照，重复恢复对应恢复成功和成本，361ms 动作块对应连续执行的停顿统计，DeepSeek 请求长尾对应固定请求优化。融合训练的价值，也用这些已经有基线的数据来检验。〔E3，实验设计〕

附录 A PPT 内容拆分

以下按报告顺序组织为 18 个证据单元，每个单元给出 1–3 张可选内容。正文页承担实验比较和结论，备份页保留逐集、逐请求或测量细节；制作时可根据汇报时长合并或展开，总页数由内容取舍确定。表注沿用报告中的批次、样本范围、计时定义、来源编号与证据等级。

A01 测评基准与近期进展

对应正文 1.1

第 1 张 正文。用一张任务分类表并列 RoboDojo 与 LIBERO：RoboDojo 的 42 个仿真任务按 12/6/8/8/8 分成五类，另列 18 个实机任务；LIBERO 列 Spatial、Object、Goal 和长任务的代表问题，并标出本地 bowl、GPT-as-Policy、RPent 所处位置。保留抓取、窄孔插入和多阶段操作三个示例画面。

讲解重点：先说明不同实验分别在测定位、状态判断、接触执行还是长任务，再阅读后面的成功数。

第 2 张 正文。用按加入日期排列的表展示官网快照：Astra 28.97/22.48%、PhysicalRSI 36.27/31.38%、VPP2 39.26/32.26%、EmbodiedEvo 45.08/38.95%，两列分别为 Score/SR。旁边简注策略模型与 Agent 系统的提交类型，页脚写 2026-10-16 访问日期。

讲解重点：近期进展既来自模型，也来自记忆、代码和反馈的组织方式；本报告接下来分别检查这两部分。

A02 bowl 同初态比较与 Astra 恢复过程

对应正文 2.1、2.2

第 1 张 正文。以 9 月 23 日同 seed 表为主体：Astra、Sol medium、Sol ultra、Luna 分别 5/5、2/5、3/5、0/5；同步放平均决策数 16.0/18.2/17.6/19.6 及墙钟 336.11/547.09/600.63/1969.76s。右侧小图补 9 月 16 日三家基线 48/60 停在阶段 0–1，表注保留自动入碗与录像审核定义。

讲解重点：Astra 的优势既表现为完成更多任务，也表现为较少的决策和等待；增加 Sol 思考预算改变了轨迹，并未在每个初态都改善结果。

第 2 张 正文。画 trial-03 的五步观测序列：实测未到位→抓空试举→看到物体留在桌面→抓取高度由 0.825 降到 0.805m→确认随手移动后搬运；保留 13 次决策、447 步。并排放 DeepSeek 误判持物与 Qwen 成功验证随动的简短案例。

讲解重点：差距来自如何使用动作后的新证据：确认实际状态，识别抓空，再选择修正动作。

A03 从轨迹确定模型诊断对象

对应正文 2.3、2.4

第 1 张 正文。用四行表对应定位、状态核验、时序适配、恢复选择，分别列配对输入改动与读数。右下角用 RPent 总成功 50/60 与 49/60 作转场，注明模型在 RPent 改为选择技能。

讲解重点：先用诊断实验找出误差来源，再训练定位、抓持确认或恢复；进入技能系统后，模型承担的工作会改变。

A04 18 格实验 三种控制分工怎样失败

对应正文 3.1、3.2

第 1 张 正文。上半页画 Student、Direct、Hybrid 三条同尺度流程；下半页放六条件×三路线的原生成功与步数矩阵。汇总写 4/6、1/6、3/6，重点框出 Pos 六格原生均失败、Goal Ori Hybrid 零步失败。展示裁定与原生判据的差异放表注。

讲解重点：相同初态下，动作由谁生成、审核怎样使用工具，会直接改变失败类型。

第 2 张 备份。并列两个轨迹：Hybrid 把 stop 当观察接口，原跑与复跑均零步结束；Direct 在换目标任务第 461 步才发现包装混淆，余 59 步。保留同条件 Student 284 步成功、Hybrid 416 步成功且 84 轮全放行，以及 Goal Pos 中心距 36.75mm 和 30mm 阈值。

讲解重点：接口终止语义、对象绑定和放置判据要分别检查，才能选择对应的修复。

A05 九回合 审核增加多少等待 改变了哪些动作

对应正文 3.3

第 1 张 正文。以三任务×三路线的结果／控制步／轮数表为主体，旁边画双罐单轮耗时：Student 4.203s、Direct 14.413s、Hybrid 18.818s，分别标出后两者 10.819s 和 14.374s 的 GPT 往返。注明双罐与微波炉 Hybrid 各 51 轮全部放行。

讲解重点：这两条成功轨迹中，逐块审核主要增加串行等待，下一步先测试紧凑放行响应。

第 2 张 正文。左侧保留 Hybrid 平均 331.8 个可见 Token、9.62s 输出窗口，画完整审核 JSON 与最小放行字段。右侧放抽屉 47 次放行、13 次接管及中层–0.159787m、底层 0 的结果；以目标部件前后图说明如何评价接管效果。

讲解重点：压缩输出解决固定响应成本，目标与进展核验解决接管是否有效，两项分开对照。

A06 RPent 普通 Flash 与失败重试

对应正文 3.4、3.5

第 1 张 正文。一张配置与结果表并列普通 API 和 Flash：GLM/SAM3 与固定计划/Molmo2，动作端均保留 π 与解析工具；首跑 18/20 对 15/20。时间列分别注明 API 全 20 集 890.20s、成功 18 集 619.22s，Flash 成功 15 集 228s。右侧将 Flash 五次失败分为两次计划名称错位、三次执行后未达目标。

讲解重点：固定流程省去在线规划，计划绑定和动作后的检查决定它能覆盖哪些场景。

第 2 张 正文。画两组首跑→retry-once 成功数：Object Swap 18/20→20/20、goal 16/20→19/20。下方并列早停回测省 21.5% Token 与截断 5 个成功集，保留 823s 失败→4795s 成功这一轨迹。将新增 16.9M Token／5 次成功列为重试成本。

讲解重点：高消耗常常表示任务困难；恢复决策需要结合动作是否推进，而不是只看请求数。

A07 长任务反馈怎样改 JEV 放在哪个环节

对应正文 3.6、3.7、3.8

第 1 张 正文。先放 Long Swap 11/20 与 12/20，再用 t0 的 15/15/28/116 个 π 块和对应成功／成功／成功／失败说明局部结果差异。画抓杯→送入→释放撤手→关门四阶段，在每段下列物体随动、空间关系和门运动的观察证据。

讲解重点：工具返回需要告诉高层哪个阶段推进了、哪里需要恢复，复合技能才有可靠的完成与退出条件。

第 2 张 正文。用一张递进对照表列状态验收、局部恢复、复合技能和固定候选选择器；选择器一行明确规则/Astra/JEV 使用同状态同候选。侧栏写本地规则 9/9、94 项测试通过与 Jev/MiniCPM 路径核查。底部给一条含适用条件、检查点和失败证据的 Skills/Lessons 示例。

讲解重点：先确认状态与候选是否正确，再比较谁选择得更好；JEV 接在技能选择位置，π 继续负责接触动作。

A08 120 集效率基线与成功产出成本

对应正文 4.1

第 1 张 正文。按三个套件画成功数与回合均时两组并列柱，保留 Astra 50/60、Sol 49/60。Long Swap 重点列 19474.4/32601.1s 全批工作量与 11/12 次成功，换算每成功产出 1770.40/2716.76s。

讲解重点：成功数接近时，失败尝试和长回合仍会造成明显的实验成本差异。

第 2 张　备份。完整保留六行套件×模型表：成功数、均值、p50、p95、平均计量响应数。框出 Spatial Task Sol 的均值 1248.62s、p50 811.60s、p95 3888.39s；表下注明全 120 集 123408.4s 工作量与 5014 个计量响应。

讲解重点：均值、尾部和调用量共同解释成本，便于追查少数长回合。

A09 同任务配对与请求 执行两侧的耗时差

对应正文 4.2

第 1 张　正文。左侧画 Spatial Swap 共同成功 20 对的总耗时 11738.4/18873.7s 及 37.81%差额；右侧把 7135.3s 差额分为 SDK 非工具 46.56%、手工与 VLA 原语 43.66%、工具其余 9.78%。下方列 579/865 次计量响应、186/286 个动作、175/407 个 π 块。

讲解重点：相同任务都完成时，差额同时来自更多请求与更多动作，优化需要覆盖两侧。

第 2 张　正文。并列两项分解：SDK 非工具差额 3321.863s 中，调用数项 2315.970s 占 69.72%，均摊成本项 1005.893s 占 30.28%；Long Swap 全 20 对差额 13126.7s，共同成功 8 对只占 536.4s，其余 12 对占 95.91%。小表保留 120 集五类互斥时间占比。

讲解重点：少调用在当前请求差额中占很大比重；Long Swap 进一步要追查失败与分叉路径。

A10 八集深测 渲染 π 与重复恢复各花了多少时间

对应正文 4.3、4.4

第 1 张　正文。画八集工作量分解，突出渲染 4391.179s/40.05%、物理 121.481s/1.11%、π handler 198.653s/1.81%；将 prefix 96.186s 和 denoise 86.142s 画在 handler 内部。侧栏放 π 减半仅节省整批 0.91% 的条件预算。

讲解重点：提高当前仿真吞吐先检查视觉观测生成与读回，π 内部优化另看实际热点和交付时限。

第 2 张　正文。用 t0/seed1 两行时间轴并列 Astra 成功 1049.1s 与 Sol 失败 2176.5s，标 41/77 次逻辑采样、15/116 个 π 块。放大 Sol 第 18–34 动作：执行累计 562.942s，其中渲染 447.543s；该段时间轴跨度 1197.889s。请求、工具、渲染使用各自泳道。

讲解重点：重复抓取同时增加云端等待、本地动作与渲染；仅缩短一次 π 推理无法消除整段恢复长尾。

第 3 张　备份。保留固定观测微测 none/stages 中位 361.047/363.719ms、各 3 次、0.74%插桩增量和动作字节一致；旁列 120 集加载 20817.6s、独立深测两次 OOM 尝试 3205.2s。表注分别说明稳态、启动与异常尝试的分母。

讲解重点：部署成本包含稳态推理、启动和异常恢复；固定输入微测提供后续本地优化的可对照起点。

A11 DeepSeek 档位变化如何改变单次与整集成本

对应正文 5.1

第 1 张　正文。完整保留四条共同成功配对：非流式 t0/s0 1000.9→1432.6s、s1 966.8→769.7s；流式 s0 1497.8→1043.1s、s1 828.9→805.1s。每行同时放逻辑请求和工具调用变化，重点框出 26→46 请求、46→102 工具的反例。

讲解重点：降低思考预算的效果取决于动作路径；单次均摊下降 4.77%，仍可能因调用增多使整集更慢。

第 2 张　备份。把原始非流式 8 集和流式补测 8 集分成上下两表，逐行保留 task/seed、档位、结果或退出状态、回合时间、逻辑请求/尝试、工具调用。另列流式 t0/s1 累计请求省 94.495s、其余成本增加 70.695s、整集仅省 23.8s 的分解。

讲解重点：请求数相同也可能走不同动作路径，解释收益时要同时保留工具成本和退出状态。

A12 有效 TTFT TPOT 和可见 thinking 的分段结果

对应正文 5.2

第 1 张　正文。上方画请求发出、首有效 delta、首正文/工具、最后 chunk 的时间线，标 TTFT、可见 thinking 和输出窗口；下方放两档四项分布：完整请求 p50/p95 为 17.371/40.855 与 12.272/21.290s，TTFT 为 11.891/21.359 与 10.595/16.893s，TPOT 为 5.667/14.171 与 4.374/7.105ms。

讲解重点：事件打点说明等待发生在哪个窗口，按 Token 归一的 TPOT 说明请求内的平均输出速度。

第 2 张　正文。画四集累计请求 1954.255s 的构成，突出 TTFT 1383.998s、占 70.82%。旁边按 medium/none 列可见 thinking 376.252/0s，正文工具 73.677/107.915s；输出组成再列 45618 个 reasoning 和 16217/23370 个其他 Token。

讲解重点：这组任务主要等首输出；关闭思考后，正文与工具输出仍会变化，应同时优化输入等待和响应协议。

第 3 张　备份。用一张数据身份表说明 306 条原非流式、178 次流式尝试/177 个有效 TTFT、精确映射代理 280 行，共 458 行/23 组、457 行有效 TTFT。另列事件 thinking 376.252s 与按 Token 占比估算 352.550s；prefill 单列服务端 trace 测点，当前记录栏写未采集。

讲解重点：数据身份、事件窗口和预算估算分别保留，后续服务端测量可直接补到相应阶段。

A13　v5 十五次请求 两种长尾如何分别处理

对应正文 5.3

第 1 张　正文。保留全部 15 根请求堆叠柱，颜色对应 TTFT、可见 thinking、正文/工具；页首列请求累计 93.592s、回合 182.5s、请求 p50/p95 3.548/18.924s。框出第 11 次 24.546s 和第 14 次 16.514s，合计占请求时间 43.87%。

讲解重点：v5 的主要等待已转到首输出之后，少数长请求决定了优化优先级。

第 2 张　正文。并列第 11 次和第 14 次明细：第 11 次 thinking 19.959s/4491 reasoning Token；第 14 次正文工具 8.939s/2687 其他 Token、两次 write_text_file。下方拆 v5 TTFT：前处理 9.930s、响应头前 10.415s、头后首输出 6.907s，并列全局 thinking 47.144s。

讲解重点：长思考适合比较预算与判断任务，长记录输出适合压缩字段并检查能否移出动作等待。

第 3 张　备份。保留 15 行完整请求表：总时长、TTFT、thinking、正文工具及两类 Token；旁列 ds_t2_par 同为 15 请求但累计 110.534s、TTFT 21.739s、首输出后 88.795s。另列第 11 次 thinking 减半可省 9.980s 的条件预算。

讲解重点：相同请求数可以具有不同输出负载，优化应落实到具体长尾请求。

A14　Files API 历史上下文与缓存分别改变了什么

对应正文 5.4

第 1 张　正文。画 v5 三条独立序列：请求字节、上下文 Token、缓存占比。标平均载荷 3136.1→185.7KB、末轮 10118→280KB，上下文 13094→84686 Token；汇总写 780723 输入、720256 缓存、60467 未缓存，按 Token 加权缓存率 92.25%。

讲解重点：文件引用减少序列化载荷，历史长度和新增输入仍需分别优化。

第 2 张　备份。并列两个独立单局：v5 为 15 请求/182.5s 回合/93.592s 请求/27.256s TTFT；Files API 局为 11/153.1/100.5/57.762。Files 局补请求 p50 6.8s、TTFT p50 3.649s，thinking/decode 估算 23.5/19.3s，表头写明事件测量与估算。

讲解重点：比较传输方式时，应固定同一请求体并交替发送，记录首次上传与重复引用的不同成本。

A15 固定请求重放与载荷阶梯怎样缩小归因范围

对应正文 5.5

第 1 张　正文。画四个固定请求体组的 TTFT 范围点图：13372 Token 为 2.052–2.671s；64652 为 9.070–17.138s；81748 为 10.861–14.971s；229734 为 17.648–23.447s。图上保留各组次数 5/5/5/3 和 p50 2.176/11.974/14.511/21.490s，侧注总计 9 体 42 次发送。

讲解重点：较大输入伴随更长等待，同一输入内部也有明显波动；下一组对照需交替发送并按缓存状态分层。

第 2 张　备份。分别展示载荷阶梯与输入/输出阶梯。载荷约 0.05/0.5/1.5/4.3MiB 对应最短 1.386/2.369/2.445/3.855s，明确标 404 网关测量；输入斜率按中位与最小值各列两组，输出中位斜率 1.391546ms/Token；补 16 次 stream/plain 的工具参数验收项。

讲解重点：字节成本、输入规模和服务波动已有独立观测，服务端排队、batch 和 kernel trace 继续补齐计算归因。

A16 把已有失败转成模型与 Harness 的下一轮对照

对应正文 6.1、6.2

第 1 张　正文。用四行实验结果→训练样本→验收指标表：抓空搬运、抽屉目标错误、重复 pick、Astra 重抓恢复。列基座、状态 SFT、决策蒸馏、RL 四个版本，明确固定执行器；输出成功、假抓持/假完成、恢复成本三组读数。

讲解重点：把训练集中到实际决定失败的状态与修正动作，逐项检查能力增量。

第 2 张　正文。用 H0–H4 表列当前 RPent、已校准状态、局部恢复、带检查点的复合技能、事件触发；每行对应请求、π 块、恢复时间、漏检和终局成功。旁列固定状态与候选的规则/Astra/JEV 对照，并将经验表示作为独立实验轴。

讲解重点：先测状态是否准确，再测恢复和调用粒度，每次新增机制都记录它增加的本地成本。

A17 协作频率 low think 与单次请求优化

对应正文 6.3、6.4

第 1 张 正文。保留 20Hz 目标下 K=2/5/8/10 的 100/250/400/500ms 覆盖表，减去 361.047ms 后预算为 −261.047/−111.047/38.953/138.953ms。用缓冲动作与下一块推理的时序图解释提前推理，并单列 p95/p99、观测年龄、缓冲耗尽和截止失约。

讲解重点：动作块是否接得上由覆盖时间和完整返回延迟决定，动作越长也越晚使用新观测。

第 2 张 正文。主图列导师最新 thinking 4.6→1.4s、减少 3.2s/69.6%，标 E4 转述；旁边单列独立校准 medium/low 完整请求中位 8.949/7.463s。下方画同请求体交错测试→同初态任务两步流程，保留完整延迟、选择正确、调用数和恢复成本。

讲解重点：先测 low think 缩短了哪段时间，再看任务中是否增加调用或恢复。

第 3 张 备份。列三组待测配置：K 扫描与同步/提前推理；高层事件触发与 0.2/0.5/1Hz 本地或延迟注入扫描；视觉编码、prefill、decode、KV 及 π prefix/denoise 的固定输入 profile。每组列固定条件、延迟测点和输出验收。

讲解重点：先用固定输入定位单次热点，再用阶段和任务测试确定采用的频率与执行方式。

A18 融合架构与 Motus 联合后训练

对应正文 6.5、6.6

第 1 张 正文。画 VLM 状态/子目标→规则或 JEV 选择→VLA 与解析控制执行的主路径，WAM 在旁路预测候选后果并与真实下一状态比较；用四行表说明各模块的输入、输出和独立评价。底部以三列区分 low think 调整单次预算、JEV 改变选择环节、VLA 分层改变高层介入次数。

讲解重点：按已出现的错误分配工作，先逐模块验证，再测试组合是否减少失败与总等待。

第 2 张 正文。先列 Motus 工程结果：约 8.06B 基座、43,502,271 适配参数、双卡 FP32 十步与精确恢复、每卡 57.34/59.47GiB allocated/reserved。再画 A 全部冻结、B 动作/理解连接、C 加 VLM LoRA、D 加 WAN LoRA 四臂，同起点同预算；并排保留控制成功与固定动作预测质量两张结果表的位置。

讲解重点：现有工程验证支持开展联合训练对照，下一步用任务表现和预测质量判断各模块更新的作用。

第 3 张　备份。保留工程配置、两路图像与真实 VAE/T5 输入、step9 恢复重做 step10 的验收项；训练核对表列数据时间对齐、初态隔离、梯度归属、冻结/遮断梯度对照、BF16 数值与任务 RL 结果。

讲解重点：工程更新、恢复一致性和任务能力分别留证据，便于解释联合训练中的增益与退化。

附录 B　数据与来源

data 目录保留 15 张脱敏 CSV；derived-analysis.json 记录本版新增配对与耗时分解。table-data.json 和 figure-data.json 保存正文图表编号，ppt-plan.json 保存独立的演示拆分。统计分位数采用线性插值，新增复算包含闭合检查。

L 为本地发表报告，S 为本次精细计时使用的数据或采集定义，W 为外部论文与官方项目。外部榜单快照核验日期为 2026 年 10 月 16 日。

本地实验报告

L01　具身智能测试项目与数据汇总 v1.3

L02　bowl 多模型总结与同 seed 对照

L03　GPT-as-Policy 三任务逐轮决策实录

L04　RPent Codex 双模型 120 集结果

L05　RPent 120 集运行账本与性能报告

L06　RPent 独立 8 集精细计时数据

L07　DeepSeek 两场景性能审计与分阶段表

L08　DeepSeek 单局 v5 与 Files API 逐轮表复算

L09　EmbodiedJev 项目分析与 RPent 对比

L10　RPent 扩展 FlashMode 与 libero_goal

L11　COAP 论文分析与 RPent 实验建议

L12　EmbodiedEvo 分析与 RPent 经验复用实验建议

L13　VLM VLA 双脑架构与推理优化调研

L14　Motus 与 910B3 联合端到端 RL 预注册

L15　Motus 联合训练模型选型核查

L16　10 月 16 日完整 Motus 工程小训练记录

L17　GPT-as-Policy 18 格运行记录

L18　GPT Token 与 Prefill Decoder 规律分析

L19 RPent GLM 普通模式 Object Swap 20 集

计时证据与复算

S1 120 集逐集结果、请求计量、五类互斥时间；由安全 CSV 复算

S2 8 集、时间轴、动作和嵌套阶段；quantitative-conclusion 与 excluded-attempt-cost-ledger 交叉核验

S3 render-path-audit.json、vla-calibration.json、logical-sampling-count-convention.json

S4 audited-data.json、streaming-phases.csv、proxy-run-mapping.json、io-evidence.json

S5 HTTP 流采集/TTFT/TPOT 定义；同时读取 proxy-source-audit.md 的代理字段审计

S6 ds_flash_v5_report.html 与 files_api_report.html 的已提取安全逐轮表；不展开会话/图片

S7 stream-probe.json、upload-ladder.json、calibration.json、cliff-ab.json

外部论文与官方资料

W01 RoboDojo v1 paper
W02 RoboDojo official homepage leaderboard and News
W03 LIBERO official datasets
W04 LIBERO official repository
W05 RoboSpatial
W06 SpatialVLM
W07 DAgger
W08 Embodied-R1.5
W09 Pointing-VLA
W10 Embodied-R1
W11 Embodied Turing Machines: Stateful Code for Robot Recursive Self-Improvement
W12 EmbodiedEvo: Self-Evolving Coding Agent For Physical Intelligence
W13 Real-Time Execution of Action Chunking Flow Policies
W14 Hi Robot: Open-Ended Instruction Following with Hierarchical Vision-Language-Action Models
W15 Helix: A Vision-Language-Action Model for Generalist Humanoid Control
W16 Motus: A Unified Latent Action World Model
W17 Motus2: A Self-Evolving General World Model for Dexterous Manipulation
W18 EmbodiedJev fixed-source candidate selection and servo
W19 ReinFlow: Fine-tuning Flow Matching Policy with Online Reinforcement Learning
W20 LIBERO-PRO official repository
W21 Motus2 official code release status
W22 Motus Team official model directory